



上海大学

SHANGHAI UNIVERSITY

本科毕业论文（设计）

UNDERGRADUATE THESIS (PROJECT)

题 目： 多模态混合专家模型的推理优化

学 院： 计算机工程与科学学院

专 业： 计算机科学与技术

学 号： 22122809

学生姓名： 李耀东

指导教师： 滕中梅

起讫日期： 2026 年 1 月 19 日至 2026 年 6 月 5 日



姓 名： 李耀东

学号： 22122809

论文题目： 多模态混合专家模型的推理优化

原创性声明

本人声明：所呈交的论文是本人在指导教师指导下进行的研究工作。除了文中特别加以标注和致谢的地方外，论文中不包含其他人已发表或撰写过的研究成果。参与同一工作的其他同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

签名：_____ 日期：_____

本论文使用授权说明

本人完全了解上海大学有关保留、使用学位论文的规定，即：学校有权保留论文及送交论文复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容。

（保密的论文在解密后应遵守此规定）

签名：_____ 指导教师签名：_____ 日期：_____

摘要

近年来，多模态大模型与混合专家模型（Mixture of Experts, MoE）结合后，在视觉问答、文档理解与复杂图文推理等任务上表现出较强能力，但其推理阶段同时面临视觉前端开销大、专家路由动态性强以及显存受限部署困难等问题。针对这一背景，本文围绕 VL-MoE 推理优化展开研究，重点从视觉侧和专家侧两个方向提升系统效率。

在视觉侧，本文提出面向多模态推理的语义感知视觉 Token 压缩与异步流水优化方法。该方法利用文本语义对视觉区域进行相关性打分，并根据语义分布的集中程度自适应调整压缩率，在保留关键信息的同时减少冗余视觉计算；同时将视觉编码拆分为可并发执行的阶段，与文本侧准备过程形成重叠，以降低首字延迟。

在专家侧，本文重新采集 Qwen3-VL 的 fresh 路由 trace，构建包含 layer-0 router 概率、Top- k 专家、token 位置、输入 id 与模态标识等特征的全局预测数据集，并训练 layer-0 条件化的全局路由预测器。该预测器在验证集上的 best mean recall 达到 0.7219。基于预测结果，本文进一步设计了 ExpertFlow 风格的加权预取、DALI 风格的 working-set 重排以及 Hybrid 融合调度策略，用于在 CPU-GPU expert offload 场景下提前加载高概率专家、降低同步等待和传输开销。

实验结果表明，视觉侧优化在 MMBench 与 MME 子集上的归一化任务分数均保持在 0.995 以上，未造成明显模型效果下降；同时将 Visual Encoder 可见耗时从 8572ms 降至 800ms，使整体 TTFT 获得约 1.63x 加速。在真实 CPU-GPU expert offload 微基准上，预测式调度相较 demand 策略能够显著减少专家加载时延；在代表性设置 cap96_g12_t64 和 cap160_g20_t64 下，predictor 相对 demand 的加速比分别达到 2.99x 和 2.66x。进一步的调度对比实验显示，在 GPU expert cache 容量为 128 时，ExpertFlow、DALI 和 Hybrid 三种策略均可稳定获得约 2.52x 以上的整体加速。本文工作为 VL-MoE 在资源受限场景下的高效部署提供了可行的系统实现路径。

关键词: VL-MoE; Visual Encoder; 专家预测; 专家调度; CPU-GPU offload

ABSTRACT

Vision-language mixture-of-experts (VL-MoE) models have shown strong performance on visual question answering, document understanding, and multimodal reasoning, but their inference pipeline still suffers from expensive visual encoding, dynamic expert routing, and difficult deployment under limited GPU memory. This thesis studies inference optimization for VL-MoE from two complementary perspectives: the visual front end and the expert side.

On the visual side, a semantic-aware token compression method is introduced for multimodal inference. It scores visual tokens with text-guided relevance, adapts the compression ratio according to semantic concentration, and overlaps the visual pipeline with text preparation to reduce TTFT.

On the expert side, a fresh Qwen3-VL routing dataset is collected and used to train a layer-0 conditioned global route predictor with features such as router probabilities, top- k experts, token positions, input ids, and modality flags. The predictor reaches a best mean recall of 0.7219 on the validation set. Based on its outputs, three scheduling policies are designed: an ExpertFlow-style weighted prefetch policy, a DALI-style working-set rebalance policy, and a hybrid policy for CPU-GPU expert offload.

Experiments show that the visual-side optimization keeps normalized task scores above 0.995 on MMBench and MME subsets while reducing the visible Visual Encoder latency from 8572 ms to 800 ms, leading to about 1.63x TTFT speedup. On a real CPU-GPU expert offload microbenchmark, the predictor significantly reduces synchronous expert loading. On representative settings cap96_g12_t64 and cap160_g20_t64, the predictor achieves 2.99x and 2.66x speedup over demand, respectively. Further scheduler comparisons at GPU cache capacity 128 show that ExpertFlow, DALI, and Hybrid all provide stable speedups of about 2.52x or higher. The proposed methods provide a practical path for efficient VL-MoE deployment under memory-constrained settings.

Keywords: VL-MoE; Visual Encoder; expert prediction; expert scheduling; CPU-

GPU offload

目 录

摘 要	I
ABSTRACT	II
1 绪论	1
1.1 VL-MoE 推理优化的研究背景和意义	1
1.2 VL-MoE 推理优化国内外研究概况	2
1.2.1 国内 VL-MoE 相关研究概况	2
1.2.2 国外 VL-MoE 相关研究概况	4
1.3 本文的主要工作和章节安排	4
2 VL-MoE 推理负载特征分析与问题建模	6
2.1 VL-MoE 模型结构与推理流程	6
2.2 实验平台、模型与评价指标	6
2.3 多模态输入负载特征分析	7
2.4 Visual Encoder 侧瓶颈分析	8
2.5 MoE 路由均衡性与跨层相关性分析	9
2.6 问题建模与优化目标	12
2.7 本章小结	12
3 面向 Visual Encoder 的多模态感知优化方法	14
3.1 设计目标与总体思路	14
3.2 语义感知的视觉 Token 压缩方法	16
3.2.1 问题定义与设计动机	16
3.2.2 文本引导的语义相关性建模	16
3.2.3 基于熵的自适应压缩率设计	17
3.2.4 Token 选择与实现细节	18
3.2.5 理论复杂度分析	19
3.3 面向实时推理的异步流水优化方法	19
3.3.1 设计动机	19

3.3.2	双阶段视觉编码结构	19
3.3.3	主流与视觉流的异步重叠	20
3.3.4	特征增强与最终表示构造	21
3.3.5	同步机制与工程实现要点	21
3.4	方法实现与复杂度分析	22
3.4.1	与推理框架的集成方式	22
3.4.2	时间复杂度与空间复杂度	22
3.4.3	方法优势与局限性讨论	23
3.5	本章小结	23
4	基于全局预测的专家提前调度方法	24
4.1	问题描述与设计动机	24
4.2	数据采集与样本构建	25
4.3	全局路由预测器设计	26
4.3.1	输入特征	26
4.3.2	网络结构	26
4.3.3	训练目标	27
4.4	预测驱动的专家预取与缓存调度	28
4.4.1	加权预取策略	28
4.4.2	ExpertFlow、DALI 与 Hybrid	28
4.4.3	预测错误与回退机制	28
4.5	系统实现与复杂度分析	29
4.6	本章小结	29
5	系统实现与实验评估	30
5.1	实验设置	30
5.2	Visual Encoder 优化实验	31
5.2.1	任务效果保持性	31
5.2.2	首字延迟与视觉前端加速	31
5.3	全局预测器离线结果	32
5.4	真实 CPU-GPU expert offload 微基准	33

5.4.1 小容量与中容量场景	33
5.4.2 三层提前与容量敏感性	34
5.5 调度策略对比与预算敏感性	35
5.5.1 ExpertFlow、DALI 与 Hybrid 对比	35
5.5.2 预取预算敏感性	35
5.6 结果讨论	36
5.7 本章小结	37
结论	38
参考文献	40
附录 A 关键实现代码	41
A.1 SAT-ToMe 语义引导保留模块	41
A.2 视觉编码器阶段拆分	41
A.3 全局路由预测器结构	43
A.4 全局预测器训练与 Recall 评估	44
A.5 预测结果到专家预取集合	45
A.6 真实 offload 调度入口	46
A.7 双流 overlap 集成逻辑	48
致 谢	49

1 绪论

1.1 VL-MoE 推理优化的研究背景和意义

近年来，多模态大模型（Multimodal Large Models, MLLMs）快速发展，图像、文本、语音等多种模态的统一建模逐渐成为大模型研究的重要方向。以 Qwen-VL、Qwen2-VL、MiniCPM-V 和 DeepSeek-VL2 为代表的开源多模态模型在视觉问答、文档理解、图文检索和复杂推理等任务上展现出较强能力^[1-4]。随着模型规模和输入复杂度不断提升，多模态推理系统已经从“能否部署”逐步转向“如何高效部署”的阶段。相比纯文本大模型，多模态模型在推理时不仅要处理文本序列，还需要完成视觉特征提取、跨模态对齐以及自回归生成等多个环节，因此其时延、吞吐和显存开销问题更加突出。

混合专家模型（Mixture of Experts, MoE）通过稀疏激活机制在保持总参数规模较大的同时降低单个 Token 的有效计算量，已成为提升模型参数效率的重要技术路线。将 MoE 结构引入视觉语言模型后，形成了 VL-MoE（Vision-Language Mixture-of-Experts）这一具有代表性的多模态稀疏架构^[5]。如图 1.1 所示，VL-MoE 的典型推理流程包括视觉编码、模态融合、路由决策、专家计算与文本解码等阶段。在这一过程中，视觉侧和专家侧共同决定了系统性能上限：一方面，图像通常会被划分为大量视觉 Token，导致 Visual Encoder 前端计算负担较重；另一方面，MoE Router 需要为不同 Token 动态选择专家，进而带来权重访问、缓存命中、通信调度以及负载均衡等一系列系统问题。

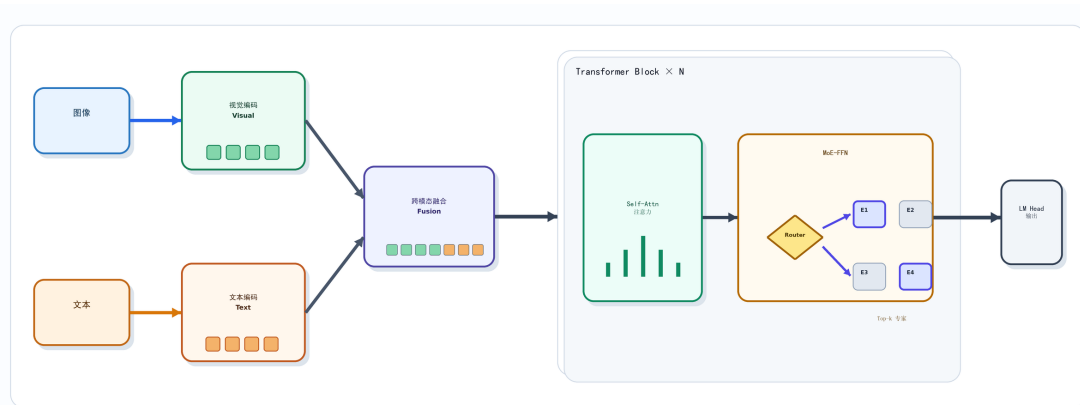


图 1.1 VL-MoE 模型的一般推理链路与专家路由示意图

从系统角度看，VL-MoE 推理优化至少面临以下三方面挑战。第一，视觉输入通常远长于文本输入，多模态请求在前端阶段会产生显著高于文本侧的计算与显存开销，导致首字延迟（Time To First Token, TTFT）偏高。第二，图像 Token 与文本 Token 在语义结构、时序特性和路由分布上存在明显差异，如果仍然套用面向纯文本 MoE 的统一调度策略，将难以充分发挥多模态稀疏模型的潜力。第三，MoE 推理过程中的专家激活具有动态性，若专家权重不能及时预取或缓存，将造成频繁的数据搬移和调度等待，尤其在资源受限或涉及 CPU-GPU offload 的场景下更为明显。

基于上述背景，围绕 VL-MoE 的推理负载均衡与加速开展研究，具有明确的理论意义和工程价值。在理论层面，通过对视觉 Token、文本 Token 以及专家路由行为进行系统化分析，可以揭示多模态稀疏模型的负载特征和跨层可预测性规律，为后续优化方法提供依据。在工程层面，通过分别从 Visual Encoder 和 MoE 调度两个关键环节入手，有望在保证模型效果稳定的前提下有效降低 TTFT、提升系统吞吐并缓解显存压力，进而提高多模态大模型在在线服务和资源受限部署场景中的实用性。

本文的研究思路并不是将优化工作割裂为多个局部技巧，而是围绕完整推理链路开展协同设计：首先通过 profiling 分析识别负载瓶颈，再针对视觉侧高耗时问题设计多模态感知的 Encoder 优化方法，最后结合专家激活的跨层相关性构建基于可预测性的提前调度机制。本文的总体技术路线如图 1.2 所示。

1.2 VL-MoE 推理优化国内外研究概况

围绕 VL-MoE 推理优化的相关研究，现阶段主要可以归纳为三条技术脉络：其一是多模态模型结构设计，重点关注如何在视觉与语言之间建立统一表示并提升模型能力；其二是视觉侧 Token 压缩与 Encoder 加速，重点解决多模态输入带来的前端高计算开销问题；其三是 MoE 推理系统优化与专家调度，重点关注专家访问、缓存、卸载以及跨设备调度效率。总体来看，国内研究在多模态模型开源和工程落地方面进展迅速，国外研究则在通用推理系统、视觉 Token 精简和 MoE 专项优化方面形成了较为系统的方法体系。

1.2.1 国内 VL-MoE 相关研究概况

国内在多模态大模型方向起步较早，并逐步形成了从模型能力构建到部署实践的完整研究链条。阿里巴巴提出的 Qwen-VL 通过统一视觉语言建模框架兼顾图文理

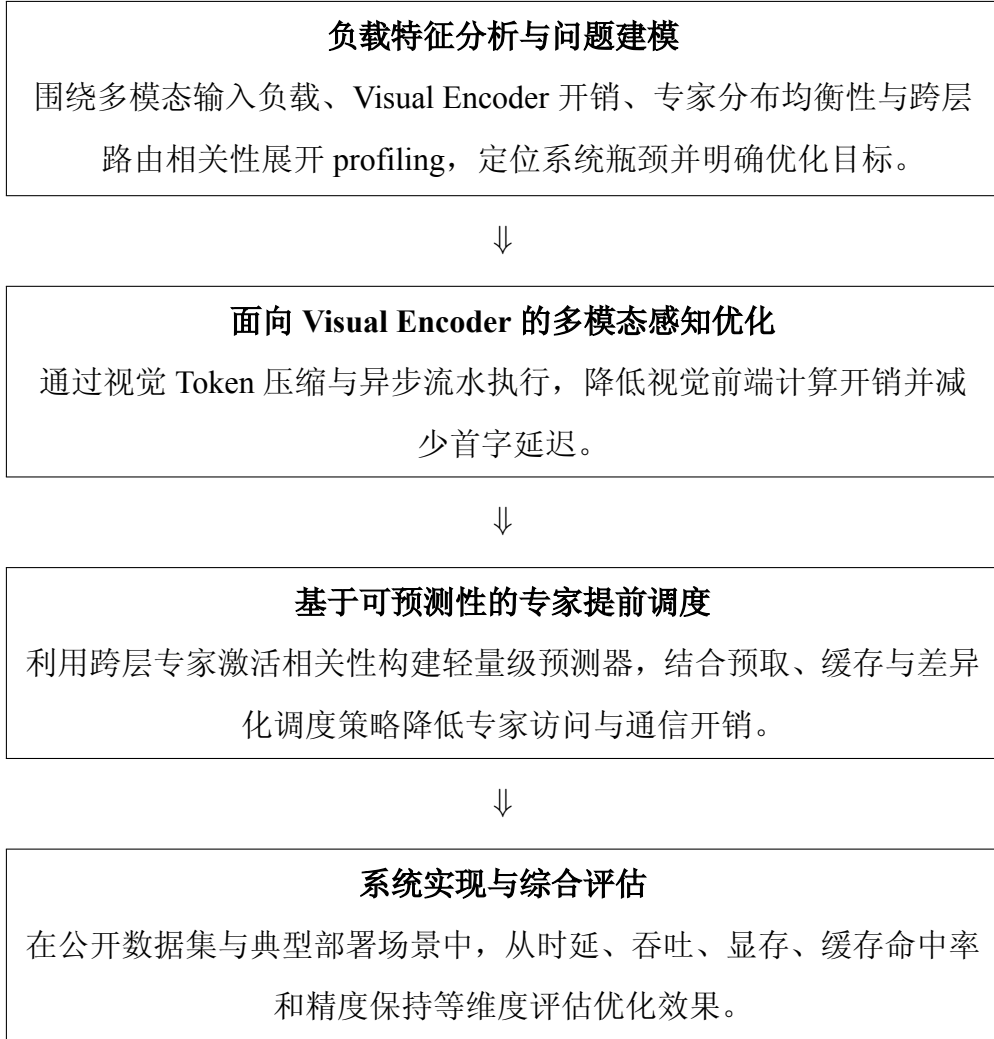


图 1.2 本文的总体技术路线

解、定位与 OCR 等任务能力^[1]；Qwen2-VL 进一步提升了模型对动态分辨率、多尺度输入和复杂视觉场景的感知能力^[2]。面向轻量化部署需求，MiniCPM-V 通过压缩与蒸馏等策略实现了终端侧多模态推理^[3]。这些工作推动了多模态模型从能力展示走向实际应用，为后续推理优化研究提供了良好的模型基础。

在稀疏化多模态模型方面，DeepSeek-VL2 将 MoE 机制系统性地引入视觉语言建模，说明 MoE 已逐渐成为多模态模型的重要结构选择^[4]。国内相关研究整体上更强调模型能力、数据构建以及部署可用性，而对推理阶段的细粒度系统剖析和底层调度优化关注相对不足。尤其是在 Visual Encoder 与 MoE Backbone 的协同优化、多模态 Token 路由差异建模、以及基于跨层可预测性的专家提前调度等方面，仍缺少针对 VL-MoE 特性的系统化方案。因此，围绕 VL-MoE 推理过程开展面向负载均衡与系统加速的研究，具有明显的必要性和提升空间。

1.2.2 国外 VL-MoE 相关研究概况

国外研究在视觉侧优化方面展开较早。Token Merging (ToMe) 通过特征相似性对 Vision Transformer 中的 Token 进行动态合并，在尽量保持模型精度的前提下显著降低视觉编码计算量^[6]。PuMer 则进一步将视觉与文本语义信息结合起来，对视觉语言模型中的 Token 同时进行裁剪和合并，验证了语义感知压缩对多模态推理加速的有效性^[7]。这类工作表明，视觉侧冗余是多模态推理优化的重要突破口。

在通用推理系统方面，vLLM 通过 PagedAttention 机制显著改善了 KV Cache 的内存管理效率，为大模型推理提供了高吞吐基础设施^[8]。DistServe 从服务系统角度将 Prefill 与 Decoding 解耦部署，以提升在线场景中的整体 goodput^[9]。上述研究对大模型推理系统具有重要意义，但其优化重点主要面向稠密模型或通用服务流程，对于 VL-MoE 特有的视觉编码链路、多模态负载差异以及专家路由异质性考虑不足。

在 MoE 推理专项优化方面，MoE-LLaVA 首次较系统地将 MoE 结构应用于大型视觉语言模型，为后续 VL-MoE 研究提供了代表性架构范式^[5]。MoE-Infinity 从激活感知卸载角度降低了专家权重搬移开销^[10]，Klotski 利用专家感知的多 Batch 流水机制提升了 MoE 推理效率^[11]。进一步地，ExpertFlow 将预测式路由路径、专家缓存与动态 Token 调度结合起来，展示了“基于可预测性进行提前调度”的潜力^[12]。然而，现有研究大多仍然面向纯文本 MoE 或通用稀疏模型展开，对 VL-MoE 场景下图像 Token 与文本 Token 的异质性利用不足，也较少从“视觉侧高开销 + 专家侧高动态性”两个问题共同出发进行端到端联合优化。

1.3 本文的主要工作和章节安排

针对 VL-MoE 推理过程中的高首字延迟、负载不均衡以及专家访问开销问题，本文围绕“特征分析、方法设计、系统实现与实验验证”四个层面展开研究，主要工作包括以下四个方面。

1. 基于 vLLM、Nsight Systems 和 PyTorch Profiler，对典型 VL-MoE 模型的推理过程进行全链路 profiling，量化 Visual Encoder、模态融合、MoE Backbone 与专家通信的开销占比，并从输入负载、专家分布均衡性和跨层路由相关性三个方面分析系统瓶颈。
2. 面向 Visual Encoder 的高耗时问题，设计多模态感知的视觉 Token 压缩与异步

流水优化方法，以减少冗余视觉计算并增强视觉侧与文本侧的执行重叠程度，从而降低 TTFT 和前端显存压力。

3. 针对 MoE 路由动态性带来的调度困难，利用跨层专家激活之间的相关性构建轻量级专家预测模型，并进一步设计面向多模态场景的专家预取、缓存管理与差异化调度策略，以降低专家访问和通信等待开销。
4. 在原型系统中集成上述优化策略，并在公开多模态数据集上从 TTFT、吞吐量、显存占用、专家缓存命中率以及任务精度保持情况等角度进行综合评估，验证方法的有效性与适用场景。

全文共分为五章，具体安排如下：第一章为绪论，介绍课题研究背景、研究意义、国内外研究现状以及本文的主要工作；第二章对 VL-MoE 推理链路进行负载特征分析与问题建模，明确系统瓶颈和优化目标；第三章研究面向 Visual Encoder 的多模态感知优化方法，重点讨论视觉 Token 压缩与异步流水设计；第四章研究基于可预测性的专家提前调度方法，给出专家预测、预取、缓存与多模态差异化调度机制；第五章介绍系统实现与实验评估，展示各项优化策略在不同场景下的性能收益与精度影响。最后在结论部分对全文工作进行总结，并讨论后续研究方向。

2 VL-MoE 推理负载特征分析与问题建模

2.1 VL-MoE 模型结构与推理流程

VL-MoE 的推理过程并不是单一的语言生成过程，而是由视觉编码、模态融合、专家路由和文本解码等多个阶段共同组成的异构执行链路。从输入形式上看，一个典型的多模态请求通常由图像和文本提示词共同构成。图像首先经过视觉编码器提取为视觉特征序列，文本则经过 `tokenizer` 编码为文本 Token 表示；随后，视觉特征与文本表示在融合模块中对齐，并送入后续的 MoE 主干网络进行联合建模；最后，模型通过自回归解码生成输出结果。

在这一过程中，Visual Encoder 和 MoE Backbone 分别承担了两类不同但同样关键的工作。Visual Encoder 的主要任务是将高维图像输入映射到后续语言模型可消费的视觉 Token 空间，其开销与图像分辨率、Patch 划分方式和视觉 Transformer 深度密切相关。MoE Backbone 则利用 Router 对不同 Token 动态选择专家，并将稀疏计算结果汇聚到主干链路中，以在有限计算预算下获得更强表达能力。然而，这种结构也天然带来了两类负担：前者是视觉侧大量 Token 导致的高计算密度，后者是专家动态激活带来的缓存、传输和调度复杂性。

从系统执行视角看，VL-MoE 推理链路可以进一步拆分为以下几个阶段：视觉预处理与编码、文本预处理、模态融合、MoE 层前向计算、专家间通信与同步、输出解码。其中，视觉侧主要影响首字延迟，MoE 专家访问则会在 Prefill 和 Decoding 阶段同时带来额外的权重调度与通信压力。因此，若要提升 VL-MoE 推理性能，不能仅仅关注某一局部算子的加速，而需要围绕完整推理链路识别负载特征和优化机会。

2.2 实验平台、模型与评价指标

为了系统分析 VL-MoE 推理负载特征，本文基于真实模型和公开数据集构建实验环境，并结合系统级 profiling 与路由级统计展开分析。实验平台与主要配置如表 2.1 所示。

本文在后续章节中主要采用以下评价指标：

1. 首字延迟 (TTFT)：衡量从请求输入到生成首个输出 Token 所需的时间，用于

表 2.1 实验平台与主要配置

项目	配置说明
硬件平台	4 × NVIDIA A6000 GPU，单卡显存 48 GB
主要模型	Qwen3-VL-30B-A3B-Instruct，以及用于对比分析的典型 VL-MoE 模型
推理框架	vLLM，部分路由统计实验基于 Hugging Face Transformers 实现 Hook 记录
分析工具	Nsight Systems、PyTorch Profiler
评测数据集	MMBench、MME 等公开多模态评测集
重点场景	Prefill、Decoding、专家缓存/预取、资源受限与潜在 offload 场景

评估前端视觉处理和 Prefill 阶段的时延压力。

2. 系统吞吐量 (Throughput)：衡量单位时间内系统可处理的请求或生成 Token 数量，用于评估整体服务能力。
3. GPU 显存占用：衡量模型在不同优化策略下的显存需求，用于分析资源受限部署场景的适用性。
4. 专家缓存命中率与专家传输次数：用于评估预测式调度和专家管理机制是否有效减少了权重搬移。
5. 任务效果指标：包括公开数据集上的准确率、得分或其他任务效果指标，用于验证优化后模型能力是否保持稳定。

上述实验设置和评价指标构成了后续分析与优化方法设计的基础。其中，第二章重点回答“瓶颈在哪里”，而第三章和第四章则分别回答“如何优化视觉侧”和“如何优化专家侧”。

2.3 多模态输入负载特征分析

与纯文本大模型相比，VL-MoE 推理的首要差异在于输入负载构成发生了根本变化。文本输入通常由长度相对有限的离散 Token 组成，而图像输入需要经过切块、嵌入和位置编码等处理后形成数量可观的视觉 Token。为了定量分析这种差异，本文

首先统计了 MMBench 和 MME 两个公开数据集上的输入 Token 构成情况。

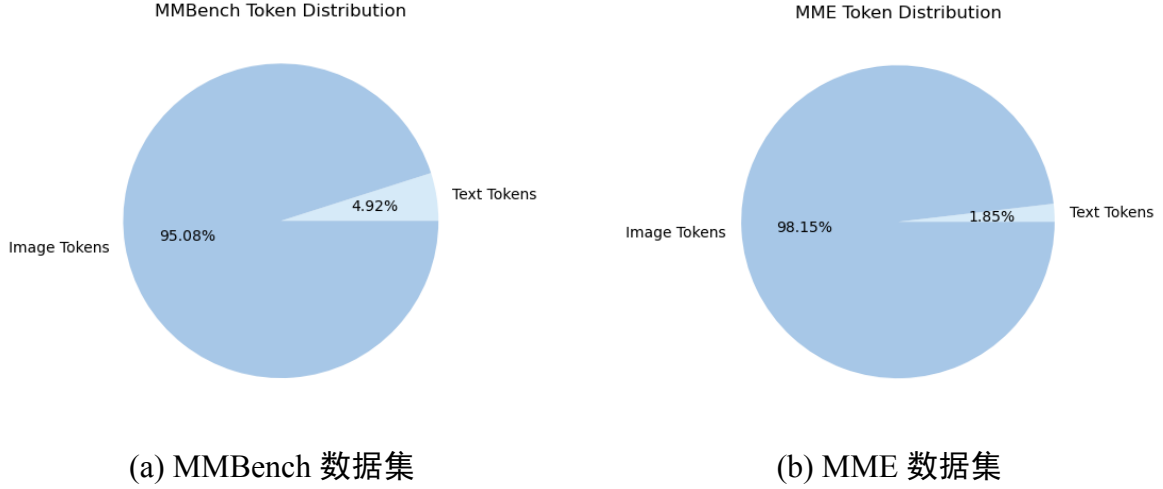


图 2.1 公开多模态评测集上的输入 Token 分布

由图 2.1 可以看出，多模态请求中的图像 Token 占据绝对主导地位。在 MMBench 数据集上，图像 Token 占比达到 95.08%，文本 Token 仅占 4.92%；在 MME 数据集上，图像 Token 占比进一步上升至 98.15%，文本 Token 仅占 1.85%。这说明，对于典型多模态推理场景而言，系统前端的大部分计算负载首先来自视觉侧而不是语言侧。

这一现象带来的直接后果是：即便模型的最终输出是文本，系统仍必须首先处理大量视觉 Token，导致 Visual Encoder 和前端融合模块在时延路径中占据较大比例。同时，由于视觉 Token 数量显著多于文本 Token，图像侧冗余区域、低信息密度 Patch 和无关背景内容也会被一并送入后续模型，进一步放大了无效计算比例。因此，从输入负载角度出发，VL-MoE 推理优化的首要任务就是识别并削减视觉侧冗余。

2.4 Visual Encoder 侧瓶颈分析

在确认多模态输入由视觉 Token 主导之后，下一步需要分析这些 Token 在推理链路中具体造成了怎样的系统压力。本文基于 vLLM 和 Nsight Systems 对端到端推理过程进行了 profiling，将执行链路拆分为 Visual Encoder、模态融合与 Prefill、MoE 专家计算以及 All-to-All 通信等阶段。

分析结果表明，Visual Encoder 是影响 TTFT 的关键前端瓶颈之一。在当前实验配置下，Visual Encoder 在基线系统的 TTFT 中占比约为 20.6%，说明即使后端采用了稀疏专家结构，视觉侧仍然是决定多模态首响性能的重要环节。其根本原因在于：视觉侧对全部视觉 Token 执行注意力与前馈计算，计算复杂度会随着视觉 Token 数

量增长而迅速上升；同时，视觉编码往往发生在生成开始之前，难以像后续解码过程那样通过流式输出掩盖等待时间。

从系统设计角度看，Encoder 瓶颈具有以下特点。第一，它主要集中在推理前端，对用户感知时延影响明显；第二，它与输入分辨率和视觉 Token 数量强相关，不同样本之间波动较大；第三，它与后续 MoE 优化并不冲突，反而是开展端到端协同优化的前提。因此，若希望显著降低 VL-MoE 的 TTFT，仅依赖后端路由优化是不够的，必须同时从视觉侧入手，减少冗余 Token 的计算并增强与文本侧执行的重叠程度。

2.5 MoE 路由均衡性与跨层相关性分析

除视觉侧开销外，VL-MoE 推理的另一个关键问题来自专家路由行为本身。为了分析 MoE 路由的系统特征，本文在 Router 处插入 Hook，对不同请求规模 and 不同层中的专家激活情况进行统计。首先考察单层内部的专家分布均衡性。图 2.2 给出了多组请求规模下专家激活占比最高的前 10 个专家分布。

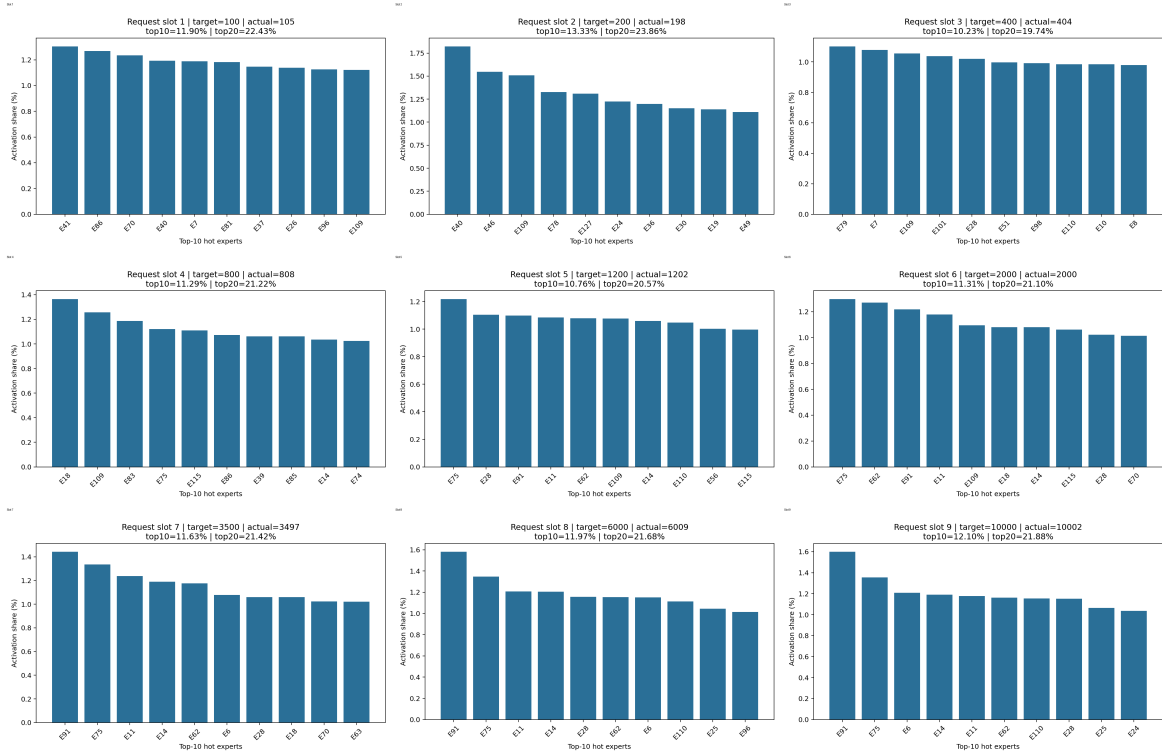
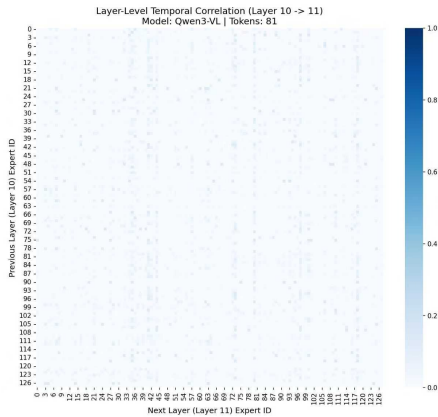


图 2.2 不同请求规模下前 10 个激活占比较高专家的分布

从图 2.2 可以观察到，不同请求规模下各专家的激活占比虽然存在差异，但整体分布并未表现出特别极端的冷热分化。前 10 个激活占比较高的专家在不同请求规模

下的占比大致稳定在 1% 左右量级，说明当前分析对象中的专家负载相对均衡，模型训练已经较好地缓解了早期 MoE 模型中常见的“少数热点专家长期拥塞、其余专家长期闲置”的问题。这意味着，对于当前这类训练较充分的 VL-MoE 模型而言，单纯依赖显式冷热专家划分开展优化，其收益空间可能会受到限制；换言之，许多以“存在非常明显热点专家”为前提的优化思路，在本文所分析模型上未必仍然是最主要的性能突破口。

然而，单层分布较均衡并不意味着专家路由完全无规律。进一步对相邻层之间的路由转移进行统计后，本文发现专家选择在层级之间仍然存在明显的结构性与路径依赖。图 2.3 给出了 Layer 10 到 Layer 11 的跨层专家转移结果。其中，图 2.3(a) 为跨层转移概率热力图，横轴表示下一层被激活的专家编号，纵轴表示前一层被激活的专家编号；图 2.3(b) 给出了基于实际激活专家对的统计结果。



(a) Layer 10 → 11 的跨层转移概率热力图

Top 比例	专家对数量	激活覆盖率
Top 1%	26/2537	8.85%
Top 5%	127/2537	24.36%
Top 10%	254/2537	36.11%
Top 20%	508/2537	51.77%
Top 30%	762/2537	61.57%
Top 40%	1015/2537	70.64%
Top 50%	1269/2537	75.54%
Top 80%	2030/2537	90.22%
Top 100%	2537/2537	100.00%

(b) 跨层激活专家对统计结果

图 2.3 相邻层之间的专家路由相关性分析

从图 2.3(a) 可以看出，虽然整体颜色较浅，说明不存在单个专家对占据压倒性优势的情况，但热力图中仍然出现了若干稳定的条纹和局部增强区域，这表明某些“前一层专家 → 后一层专家”的转移关系会更频繁地出现，而并非完全随机均匀扩散。从图 2.3(b) 的量化结果看，Layer 10 到 Layer 11 实际出现的专家对总数为 2537，而理论上可组合的专家对总容量为 16384，对应稀疏度达到 84.52%。这说明大量跨层专家对在实际运行中根本不会出现，真正活跃的转移路径只占全部可能路径的一小部分。进一步地，在这 2537 个实际出现的专家对中，前 1% 的专家对已经覆盖 8.85% 的激活，前 10% 的专家对覆盖 36.11% 的激活，前 20% 的专家对覆盖 51.77% 的激

活。这表明，尽管单层专家使用已经较为均衡，但跨层转移路径仍然具有明显的统计集中性和可预测性。

这一发现对后续优化设计具有直接启发意义。既然单层内部没有特别明显的冷热专家，那么仅围绕“哪些专家更热”进行静态缓存优化的效果会受到限制；但如果能够利用当前层或历史层的路由结果，预测下一层更可能激活的专家集合，就仍然可以提前开展专家预取、缓存调整和调度重叠，从而减少同步等待和无效传输。从多模态角度看，图像 Token 与文本 Token 在语义结构和上下文依赖上存在差异，因此它们在跨层路由路径上也可能表现出不同模式。这进一步说明，相比仅关注单层热点，基于跨层可预测性的专家提前调度更适合作为后续研究重点。

结合前文对 Visual Encoder 的分析可以看到，VL-MoE 推理瓶颈并不是单点问题，而是由前端视觉开销和后端专家调度共同构成的复合问题。前者决定了首字延迟的基线水平，后者决定了稀疏结构在真实部署中的可兑现效率。因此，后续优化设计必须同时覆盖这两个方向。

表 2.2 VL-MoE 推理中的主要瓶颈与现象总结

瓶颈类型	主要表现
输入负载失衡	图像 Token 在公开多模态评测集中的占比超过 95%，视觉侧成为主要负载来源。
视觉编码开销高	Visual Encoder 在 TTFT 中占比较高，且随视觉 Token 数量增长而显著上升。
单层专家分布较均衡	各专家激活占比整体较为平滑，没有特别极端的冷热专家分化，说明模型训练已较好缓解负载失衡。
专家路由动态性强	不同请求和不同层的激活专家集合不断变化，增加了缓存与预取难度。
跨层转移存在结构性	相邻层之间的专家转移路径具有明显稀疏性和统计相关性，为预测式调度提供了依据。
模态行为异质	图像 Token 与文本 Token 在路由模式、活跃专家和调度需求上存在差异。

2.6 问题建模与优化目标

基于以上 profiling 结果，本文将 VL-MoE 推理优化问题抽象为在精度约束和资源约束下的端到端时延最小化问题。设一次多模态请求的总推理时延为 L ，则可近似表示为

$$L = T_{\text{enc}} + T_{\text{fuse}} + T_{\text{route}} + T_{\text{expert}} + T_{\text{comm}} + T_{\text{decode}}, \quad (2.1)$$

其中 T_{enc} 表示视觉编码开销， T_{fuse} 表示模态融合开销， T_{route} 表示路由决策相关开销， T_{expert} 表示专家前向计算开销， T_{comm} 表示专家权重加载、传输和同步带来的通信开销， T_{decode} 表示最终解码阶段的开销。

在上述分解中，第二章的分析已经表明 T_{enc} 和 T_{comm} 是当前系统中最具优化价值的两个部分。因此，本文将优化目标进一步细化为以下两个方面：

1. 在保证模型输出质量基本不变的前提下，尽可能降低视觉侧冗余计算，即压缩 T_{enc} ；
2. 在不显著增加额外预测成本的前提下，提高高概率跨层转移路径对应专家的缓存命中率并减少专家搬移次数，即压缩 $T_{\text{route}} + T_{\text{comm}}$ 。

与此同时，实际部署场景还要求优化方法满足以下约束：

$$\Delta\text{Acc} \leq \varepsilon, \quad M \leq M_{\text{budget}}, \quad (2.2)$$

其中 ΔAcc 表示优化后模型效果的下降幅度， ε 为可接受误差上界， M 为运行显存占用， M_{budget} 为部署环境可提供的显存预算。

基于这一建模，后续章节的整体设计逻辑可以概括如下：第三章重点解决 T_{enc} 过大的问题，通过视觉 Token 压缩与异步流水降低前端开销；第四章重点解决 $T_{\text{route}} + T_{\text{comm}}$ 的问题，通过专家预测、预取与差异化调度降低专家访问和通信代价。两者共同服务于整体推理时延的下降与资源利用率的提升。

2.7 本章小结

本章围绕 VL-MoE 推理链路展开了负载特征分析与问题建模。首先，从模型结构与系统流程角度说明了多模态推理与纯文本推理的差异；其次，通过实验平台和评价指标的设定，为后续分析与实验奠定基础；随后，基于公开数据集和真实模型

的 profiling 结果，分析了输入负载、Visual Encoder 开销、单层专家分布均衡性以及跨层路由相关性等关键现象，明确指出 VL-MoE 推理存在“视觉侧负担重、专家侧跨层动态性强”的双重瓶颈。

在此基础上，本文进一步将优化目标归纳为两个主要方向：一是面向 Visual Encoder 的多模态感知加速，二是面向 MoE 的基于可预测性的专家提前调度。后续章节将分别围绕这两个方向展开方法设计与实现。

3 面向 Visual Encoder 的多模态感知优化方法

3.1 设计目标与总体思路

第二章的分析表明，VL-MoE 推理链路中的前端视觉处理是影响首字延迟的重要瓶颈。一方面，多模态请求中的图像 Token 数量远高于文本 Token 数量，导致 Visual Encoder 在推理开始阶段承担了大部分前端计算负载；另一方面，视觉特征提取通常发生在生成首个输出 Token 之前，难以通过后续解码过程掩盖等待时间。因此，若希望显著降低 TTFT，仅依赖后端 MoE 调度优化是不够的，必须首先从视觉侧降低冗余计算开销。

然而，Visual Encoder 优化并不能简单理解为“尽可能删除更多 Token”。对于多模态模型而言，视觉 Token 不仅承担图像内容表示，还会在后续跨模态交互中影响文本生成质量。如果压缩策略忽略文本问题本身的语义需求，就容易误删与问题强相关的局部区域，进而造成任务精度下降。由此可见，Visual Encoder 优化需要同时满足两个目标：其一，在保持关键信息不丢失的前提下尽量减少视觉 Token 数量；其二，使压缩过程与后续推理流程相协调，尽可能通过异步执行进一步压缩端到端时延。

基于上述考虑，本文提出一种面向 VL-MoE 的多模态感知视觉优化方法，其核心思想可以概括为“语义引导的自适应 Token 压缩 + 面向实时推理的异步流水执行”。其中，前者利用文本查询对视觉 Token 进行语义打分与自适应筛选，在输入进入 MoE 主干网络之前尽可能去除低信息密度区域；后者将视觉编码过程拆分为两个阶段，并利用独立 CUDA 流与文本侧处理并发执行，从而减少前端阻塞时间。整体框架如图 3.1 所示。

为了便于后续描述，设输入图像经过 Patch Embedding 和若干视觉层编码后得到视觉 Token 序列

$$X_{\text{vis}} = [v_1, v_2, \dots, v_N] \in \mathbb{R}^{N \times d_v}, \quad (3.1)$$

其中 N 表示视觉 Token 数量， d_v 表示视觉特征维度。设文本侧经过编码或聚合后得

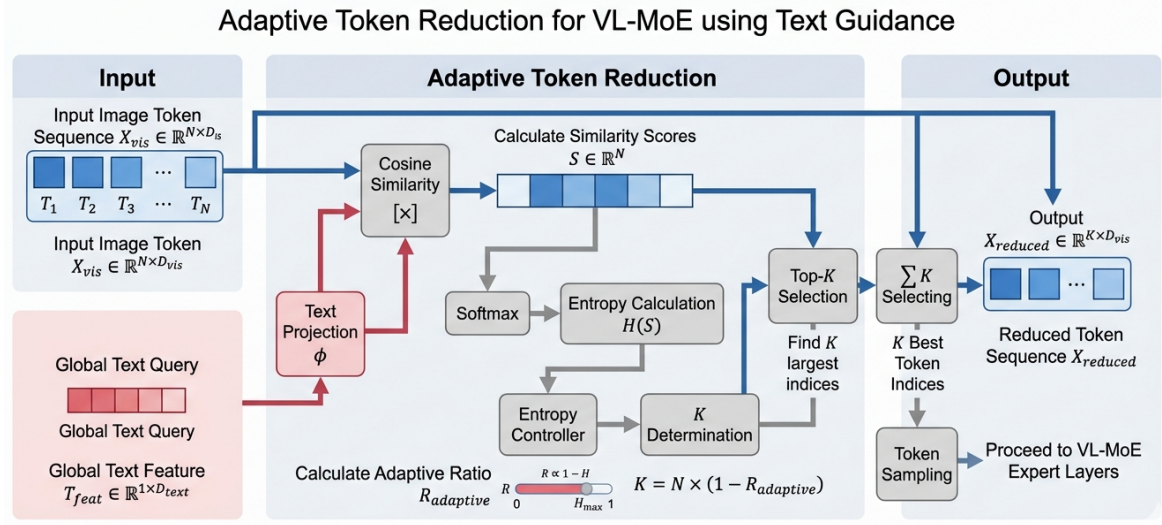


图 3.1 基于文本引导的自适应视觉 Token 压缩框架

到全局文本特征

$$T_{feat} \in \mathbb{R}^{1 \times d_t}, \quad (3.2)$$

其中 d_t 为文本特征维度。第三章的目标是在保证多模态语义对齐能力基本不变的前提下，构造更短的视觉 Token 序列

$$X_{red} \in \mathbb{R}^{K \times d_v}, \quad K < N, \quad (3.3)$$

并使其能够无缝接入后续 VL-MoE 主干网络。

围绕这一目标，本文在第三章中重点解决以下三个问题：

1. 如何利用文本语义识别当前问题真正需要关注的视觉区域，而不是做与任务无关的静态裁剪；
2. 如何根据样本语义分布的集中程度动态调整压缩强度，而不是对所有请求采用固定压缩率；
3. 如何将 Token 压缩嵌入到实时推理流程中，与文本侧计算和视觉剩余层计算形成重叠，从而进一步降低 TTFT。

3.2 语义感知的视觉 Token 压缩方法

3.2.1 问题定义与设计动机

多模态推理中的视觉冗余主要来自两个方面。第一，图像通常包含大量背景区域、重复纹理或与问题无关的局部内容，这些内容虽然会被编码为视觉 Token，但对最终回答贡献有限。第二，不同请求对图像关注的区域并不相同，同一幅图像在“数人数”和“判断场景”两类任务下的重要区域可能完全不同。因此，如果仍采用固定规则压缩视觉 Token，就难以兼顾效率与效果。

图 3.2 给出了一个直观示例：当问题为“有多少人在运动”时，模型更应保留与人物主体相关的视觉区域；而当问题变为“她们在进行什么运动”时，球和局部动作区域的重要性又会显著上升。由此可见，视觉 Token 的保留策略应当显式结合文本语义，而不能仅仅根据图像本身的静态结构进行统一裁剪或合并。

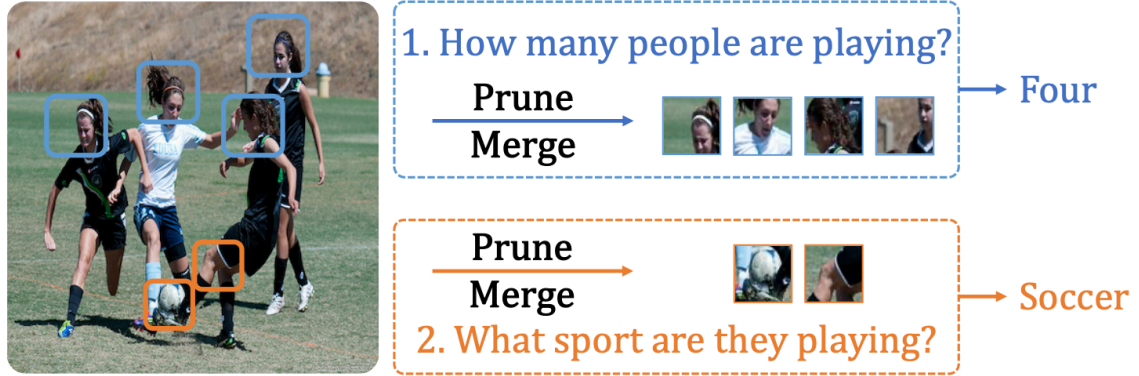


图 3.2 不同文本问题下语义相关视觉 Token 的保留重点示意图

针对这一问题，本文引入文本引导的语义打分机制。基本思想是：将文本查询映射到视觉特征空间中，测量每个视觉 Token 与文本语义的相关性，再依据语义分布的集中程度自适应确定保留 Token 数量。这样既能够优先保留与当前问题强相关的局部区域，又能够根据不同样本的语义复杂度动态调整压缩力度。

3.2.2 文本引导的语义相关性建模

为了在视觉空间中刻画“当前问题与图像局部内容的匹配程度”，本文首先将文本全局特征投影到视觉维度，得到文本查询向量

$$w_t = \phi(T_{\text{feat}}) \in \mathbb{R}^{1 \times d_v}, \quad (3.4)$$

其中 $\phi(\cdot)$ 表示线性投影层。随后，对每个视觉 Token v_i 计算其与文本查询的余弦相似度

$$s_i = \frac{w_t \cdot v_i}{\|w_t\| \|v_i\|}, \quad i = 1, 2, \dots, N. \quad (3.5)$$

由此得到语义相关性分数向量

$$S = [s_1, s_2, \dots, s_N] \in \mathbb{R}^N. \quad (3.6)$$

式 (3.5) 的含义可以理解为：若某个视觉 Token 对应的图像区域与当前文本问题高度相关，则其在投影后的语义空间中会与文本查询方向更接近，从而获得更高的分数；反之，若某个 Token 对应背景、冗余纹理或问题无关区域，则其语义得分较低。因此， S 为后续视觉 Token 筛选提供了直接依据。

3.2.3 基于熵的自适应压缩率设计

仅依赖相似度排序还不足以完成自适应压缩，因为不同请求的语义分布模式可能差异很大。例如，当问题只关注图像中的少数目标时，语义分数会集中在少量 Token 上，此时适合采用更高压缩率；而当问题涉及整体场景理解时，更多 Token 会具有中等相关性，此时过强压缩容易造成信息损失。

为此，本文将语义分数归一化为概率分布

$$p_i = \frac{\exp(s_i)}{\sum_{j=1}^N \exp(s_j)}, \quad i = 1, 2, \dots, N. \quad (3.7)$$

在此基础上定义归一化信息熵

$$H(S) = -\frac{1}{\log N} \sum_{i=1}^N p_i \log p_i, \quad (3.8)$$

其中 $H(S) \in [0, 1]$ 。当 $H(S)$ 较小时，说明语义注意力集中在少数视觉 Token 上；当 $H(S)$ 较大时，说明语义信息分散，需要保留更多视觉内容。

据此，本文定义自适应压缩比例

$$R_{\text{adaptive}} = R_{\text{base}} \cdot (1 - H(S)), \quad (3.9)$$

其中 R_{base} 为基础压缩率超参数。由式 (3.9) 可知，当语义分布更集中时， $H(S)$ 更小，自适应压缩比例更高；当语义分布更分散时， $H(S)$ 更大，系统会自动减小压缩力度。

为了避免极端情况下保留 Token 数量过少，本文进一步设置最小保留阈值 $K_{\min}$ ，并将最终保留 Token 数量定义为

$$K = \max(K_{\min}, \lfloor N \cdot (1 - R_{\text{adaptive}}) \rfloor). \quad (3.10)$$

在当前实现中，取 $K_{\min} = 16$ ，以确保模型在高压压缩率条件下仍保留足够的视觉骨架信息。

3.2.4 Token 选择与实现细节

在确定保留数量 K 后，本文按照语义分数从高到低选择前 K 个视觉 Token，得到保留索引集合 $\mathcal{I}_{\text{keep}}$ 。仅采用“直接裁剪”虽然可以减少计算，但会完全丢弃被过滤 Token 所携带的信息，容易带来边缘信息缺失和局部语义断裂。因此，本文采用“选择 + 重构”的策略：首先保留语义最相关的核心 Token，然后将被移除 Token 的信息以聚合方式补偿到保留 Token 上。

需要说明的是，从方法原理上，式 (3.11) 所示的“选择 + 重构”可以视为更完整的上限形式；但在当前工程版本中，为了降低运行时开销并便于与后续 deepstack 分支对接，SAT-ToMe 模块采用轻量化 Top-K 保留实现，直接返回保留索引 `keep_indices`，并同步更新 `rotary position embedding` 与 `cu_seqlens`。被裁剪 Token 的细节信息则主要通过后续多尺度特征分支进行补偿。相关关键代码见附录 A。

记保留 Token 集合为 $\{v_j\}_{j \in \mathcal{I}_{\text{keep}}}$ ，被裁剪 Token 集合为 $\{v_i\}_{i \in \mathcal{I}_{\text{drop}}}$ 。对于每个被裁剪 Token，按照相似度或邻近关系将其分配给一个保留 Token，并使用归一化权重进行聚合，得到重构后的保留 Token

$$\tilde{v}_j = v_j + \sum_{i \in \mathcal{A}(j)} \alpha_{ij} v_i, \quad (3.11)$$

其中 $\mathcal{A}(j)$ 表示分配给第 j 个保留 Token 的被裁剪 Token 集合， α_{ij} 为归一化聚合权重。最终得到压缩后的视觉表示

$$X_{\text{red}} = [\tilde{v}_1, \tilde{v}_2, \dots, \tilde{v}_K] \in \mathbb{R}^{K \times d_v}. \quad (3.12)$$

该设计兼顾了压缩效率与语义保持能力：高相关区域被显式保留，低相关区域的信息则通过特征聚合以低成本回流到核心 Token 中，从而降低纯裁剪带来的信息断裂风险。

3.2.5 理论复杂度分析

在视觉 Transformer 中，自注意力开销通常与 Token 数量的平方成正比。若原始视觉 Token 数量为 N ，压缩后数量为 K ，则注意力主导部分的复杂度可由

$$\mathcal{O}(N^2 d_v) \quad (3.13)$$

降低为

$$\mathcal{O}(K^2 d_v). \quad (3.14)$$

因此，仅从注意力计算角度看，理论加速比近似为

$$\frac{N^2}{K^2} = \left(\frac{N}{K}\right)^2. \quad (3.15)$$

当 K/N 下降到较小范围时，Visual Encoder 的理论开销会显著下降。与此同时，后续模态融合和部分跨模态计算也会因为视觉 Token 数量减少而进一步受益。

3.3 面向实时推理的异步流水优化方法

3.3.1 设计动机

视觉 Token 压缩可以降低单位样本计算量，但如果压缩过程本身仍然完全串行地嵌入到推理链路中，那么端到端等待时间仍然难以充分压缩。尤其在在线推理场景下，文本嵌入、视觉编码、模态融合和后续 LLM 推理原本存在一定程度的可重叠空间。若能够显式挖掘这些重叠机会，就可以在“减少计算量”之外进一步实现“隐藏等待时间”。

基于这一考虑，本文在 Encoder 优化中引入面向实时推理的双流异步流水机制。其核心思路是：将视觉编码拆分为两个阶段，在视觉流中执行视觉预处理和分段编码；同时在主流中并发执行文本嵌入与后续准备操作；在必要节点通过轻量同步保证数据一致性，从而尽可能扩大计算重叠范围。整体执行过程如图 3.3 所示。

3.3.2 双阶段视觉编码结构

为了兼顾压缩时机与实现复杂度，本文将视觉编码过程划分为两个阶段：

1. Stage 1: 完成视觉预处理以及前若干层视觉编码，生成具有初步语义结构的中间视觉表示；

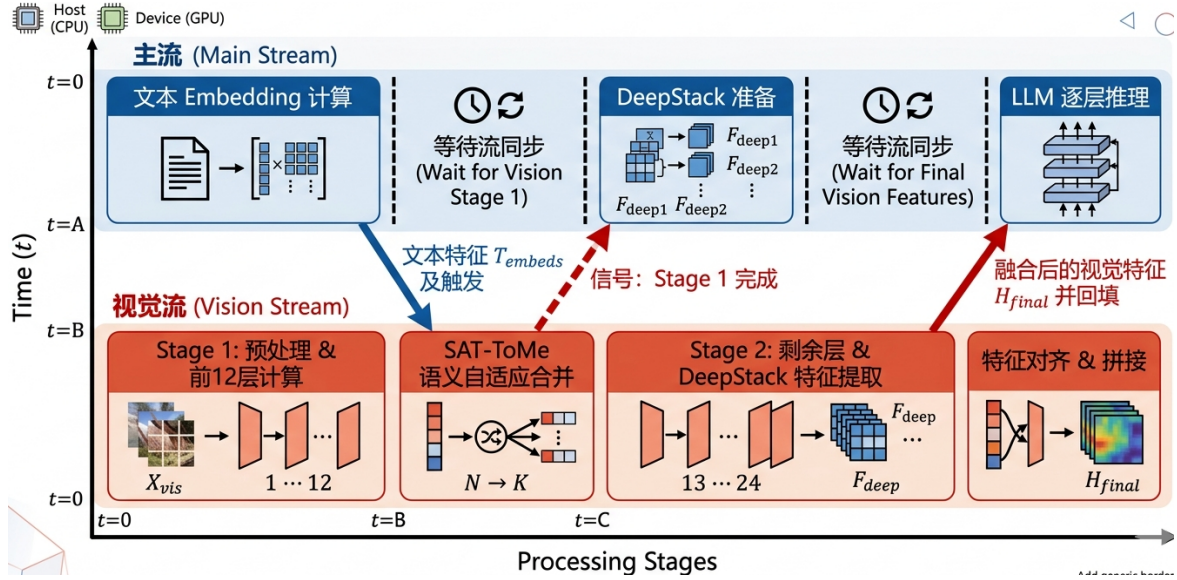


图 3.3 面向实时推理的 Visual Encoder 异步流水执行过程

2. Stage 2: 在完成 SAT-ToMe 语义自适应压缩后，对保留 Token 继续执行剩余视觉层计算，并提取多尺度增强特征。

将 Token 压缩放置在 Stage 1 与 Stage 2 之间，具有两个优点。首先，此时视觉 Token 已经过若干层编码，局部语义结构较输入阶段更稳定，文本引导打分更可靠；其次，后续剩余层的计算量更高，在此之前完成压缩可以获得更明显的时延收益。

3.3.3 主流与视觉流的异步重叠

本文使用 `torch.cuda.Stream` 为视觉侧构建独立的 `vision_stream`，使其与默认 `main_stream` 并发执行。具体而言，主流主要负责文本嵌入计算、后续多模态主干准备以及最终 LLM 推理，视觉流则负责视觉预处理、Stage 1 编码、SAT-ToMe 压缩和 Stage 2 编码。在工程实现上，原始 `forward` 被拆分为 `_prepare_vision_inputs`、`forward_stage1`、`excute_semantic_merge` 和 `forward_stage2` 四个阶段，便于在视觉流与主流之间显式插入 `overlap` 点和同步点。

在时间轴上，系统执行过程可以概括为以下步骤：

1. 主流启动文本 Embedding 计算，视觉流并发启动图像预处理与 Stage 1 计算；
2. 当主流文本特征可用且视觉流 Stage 1 结束后，触发文本引导的 SAT-ToMe 压缩；
3. 压缩后视觉流继续执行 Stage 2 计算，同时主流准备后续融合与推理步骤；
4. 在最终视觉特征准备完毕后，主流获取视觉结果并继续执行多模态主干与解

码。

该设计的关键不在于完全消除同步，而在于将同步点限制在真正需要交换信息的位置。与完全串行执行相比，大量视觉侧计算可以与文本侧准备过程并发完成，从而缩短用户感知到的整体等待时间。

3.3.4 特征增强与最终表示构造

在视觉 Token 被压缩后，模型虽然保留了主要语义区域，但仍可能丢失部分中低层细节信息。为了缓解这一问题，本文在 Stage 2 中引入多尺度特征提取机制，从若干中间层中抽取辅助视觉表示，并在最终输出阶段与主分支结果进行拼接。设 Stage 2 的最终输出为 H_{stage2} ，多尺度辅助特征为 $F_{\text{deep1}}, F_{\text{deep2}}, \dots$ ，则最终视觉表示构造为

$$H_{\text{final}} = [H_{\text{stage2}} \parallel F_{\text{deep1}} \parallel F_{\text{deep2}} \parallel \dots], \quad (3.16)$$

其中 $\parallel$ 表示特征拼接操作。

式 (3.16) 的目的并不是增加额外复杂分支，而是在较低代价下保留多尺度视觉线索，从而补偿高压缩率下可能损失的细节信息。对于文档理解、区域感知或细粒度问答等任务，这种增强策略尤其重要。

3.3.5 同步机制与工程实现要点

为了保证异步流水的正确性，本文在以下两个位置引入必要同步：

1. 在 Stage 1 完成并且文本特征就绪后，同步两条流以执行文本引导的 SAT-ToMe 压缩；
2. 在 Stage 2 与多尺度增强特征完成后，同步视觉流与主流，使最终视觉表示能够安全输入后续多模态主干网络。

此外，在工程实现中还需要处理以下问题：

1. 变长序列重建：视觉 Token 压缩后，每个样本的有效长度可能不同，因此需要重新构建 ‘cu_seqLens’ 和 ‘max_seqLen’，以适配后续高效注意力算子；
2. 缓冲区复用：为减少额外显存碎片，Stage 1 输出、中间保留索引和 Stage 2 输入缓冲区应尽量复用；
3. 与主干接口兼容：压缩后的视觉表示维度需与后续模态融合模块保持兼容，避免额外的投影或数据搬移开销。

3.4 方法实现与复杂度分析

3.4.1 与推理框架的集成方式

本文的方法在实现上遵循“尽量少改主干、尽量早做压缩、尽量充分重叠”的原则。在系统集成时，Visual Encoder 优化模块部署在视觉前端与多模态主干之间，并主要包含以下几个子模块：

1. 文本特征提取模块：负责生成全局文本查询向量，用于视觉 Token 语义打分；
2. SAT-ToMe 压缩模块：负责根据语义分数与熵计算结果动态确定保留 Token 集合，并输出保留索引供后续位置编码与序列长度更新使用；
3. 视觉双阶段流水模块：负责执行视觉流与主流之间的并发调度及同步；
4. 多尺度特征增强模块：负责从剩余视觉层中提取补充特征并构造最终视觉表示。

从接口角度看，该优化仅修改视觉编码前端和视觉特征输出形式，不改变后续 VL-MoE 主干网络的基本结构，因此具有较好的系统兼容性。对于实际部署而言，这种设计也便于逐步启用和独立评估。本章的关键实现代码整理于附录 A，正文仅保留方法说明和调度逻辑。

3.4.2 时间复杂度与空间复杂度

第三章方法的额外开销主要来自文本引导相似度计算、熵计算和 Token 聚合。设文本特征维度投影后与视觉维度一致为 d_v ，则相似度打分复杂度约为

$$\mathcal{O}(Nd_v), \quad (3.17)$$

Softmax 与熵计算复杂度约为

$$\mathcal{O}(N), \quad (3.18)$$

聚合重构复杂度与被压缩 Token 数量线性相关，约为

$$\mathcal{O}((N - K)d_v). \quad (3.19)$$

与后续视觉自注意力主导的平方级复杂度相比，这些额外开销处于较低量级，因此在理论上是值得的。

空间开销方面，方法需要额外存储语义分数向量、保留索引和少量中间聚合缓冲区，其开销近似为

$$\mathcal{O}(N + K + Nd'_v), \quad (3.20)$$

其中 d'_v 表示中间辅助缓冲区的有效维度。由于这些额外存储远小于原始全量视觉注意力缓存和后续主干状态，因此总体显存压力仍以压缩带来的节省为主。

3.4.3 方法优势与局限性讨论

本文第三章方法的主要优势体现在以下三个方面：

1. 压缩决策由文本语义直接驱动，能够适应不同问题下的区域关注差异；
2. 压缩率通过信息熵自适应调整，避免固定比例裁剪在复杂样本上带来的信息损失；
3. 通过视觉流与主流的异步重叠，不仅减少了计算量，还减少了端到端等待时间。

与此同时，该方法也存在一定局限性。首先，当文本问题本身较短或语义信息不足时，全局文本特征对视觉区域的指导能力可能受限；其次，若输入图像包含大量细粒度信息，高压缩率仍可能对精度造成影响；再次，异步流水设计依赖较强的工程实现能力，在不同推理框架中迁移时可能需要适配底层执行接口。这些问题将在第五章的实验部分通过效果对比和消融实验进一步讨论。

3.5 本章小结

本章针对 VL-MoE 推理中 Visual Encoder 前端开销较高的问题，提出了面向多模态感知的视觉优化方法。首先，本文利用文本引导的语义相关性建模，为视觉 Token 提供任务相关的打分依据；随后，基于归一化信息熵设计自适应压缩率，使系统能够根据不同样本的语义分布动态调整压缩强度；在此基础上，本文进一步将压缩模块嵌入到双阶段视觉编码和双流异步执行过程中，通过重叠视觉处理与文本侧准备来进一步降低 TTFT。

从方法定位上看，第三章解决的是第二章中识别出的第一类核心问题，即视觉侧冗余计算导致的前端高时延。后续第四章将在此基础上，继续针对 MoE 专家路由的跨层可预测性设计专家提前调度方法，与本章形成互补。

4 基于全局预测的专家提前调度方法

4.1 问题描述与设计动机

在 VL-MoE 推理系统中，MoE 层的专家访问由 Router 动态决定。对于显存受限的部署场景，专家权重往往无法全部常驻 GPU，而需要在 CPU 与 GPU 之间按需搬运。若系统等到真实访问发生时才开始加载专家，就会在专家计算路径上引入明显等待时间，从而放大 TTFT 和解码阶段时延。

围绕这一问题，本文将其定义为面向 offload 场景的**专家提前调度问题**：利用当前层 Router 已经产生的路由信号，预测未来若干层更可能激活的专家集合，并在真实访问发生前完成预取、驻留和替换决策，从而把一部分等待时间前移到当前层的计算窗口中。现有系统工作已经从 KV cache、prefill/decoding 解耦以及 expert offloading 等角度探索过 serving 优化，代表性方法包括 PagedAttention、DistServe、MoE-Infinity 和 ExpertFlow^[8-10,12]。本文进一步利用**路由可预测性**，把优化对象从“被动处理专家访问”推进到“主动规划专家访问”。

与纯文本 MoE 相比，Qwen3-VL 这类多模态模型还具有额外的条件变量。图像 Token 与文本 Token 的路由密度、位置分布和上下文稳定性并不相同，因此本文在预测器输入和调度器决策中都显式引入模态信息，而不是把它当成后处理噪声。

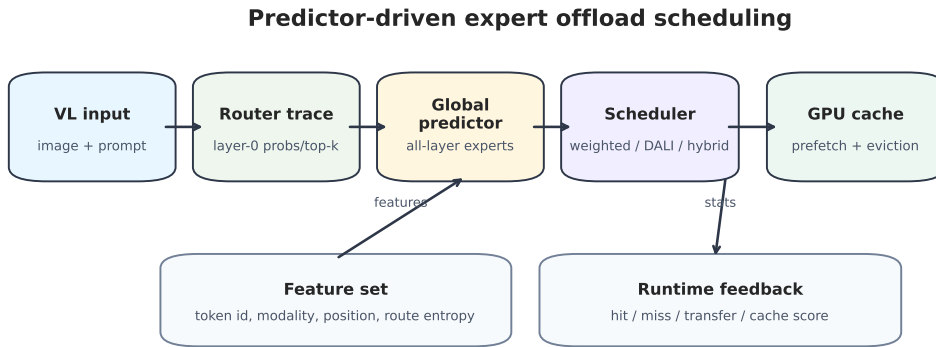


图 4.1 面向 offload 场景的全局预测与专家调度整体框架

从形式化角度看，设第 l 层输入的 Token 集合为 $\mathcal{X}^{(l)} = \{x_i^{(l)}\}_{i=1}^{n_l}$ ，Router 为每个

Token 选择 Top- k 专家集合

$$\mathcal{A}_i^{(l)} = \text{TopK}(\text{Router}(x_i^{(l)})), \quad \mathcal{A}_i^{(l)} \subseteq \mathcal{E}, \quad (4.1)$$

其中 $\mathcal{E} = \{e_1, e_2, \dots, e_M\}$ 表示全部专家集合， M 为专家总数。本章的目标是在保证预测额外开销可控的前提下，利用当前层可观测信号逼近未来层的真实激活集合，从而尽可能减少专家加载时延、数据搬移量和等待开销。

4.2 数据采集与样本构建

为了让预测器直接对真实 VL 路由模式建模，本文没有复用旧的纯文本 MoE 语料，而是重新采集了 Qwen3-VL 的 fresh 路由 trace。采集链路由三步组成：

1. 在 Qwen3-VL 推理过程中对 MoE Router 增加 hook，记录每层的 router 概率、Top- k 专家、Token 位置、输入标识和模态标识；
2. 将单次请求的逐层路由组织为 token 级样本，保留 layer-0 路由信号以及后续层监督标签；
3. 使用构建脚本将原始 trace 转换为训练所需的 Parquet 数据集，再切分为训练集和验证集。

在当前实验中，fresh 数据集共包含 8347 个 token，其中训练集 7268 个 token，验证集 1079 个 token；按模态统计，图像 token 为 6734，文本 token 为 1613。数据集覆盖了多项选择、描述、推理和图文组合等样本，因此能够提供较丰富的路由模式。

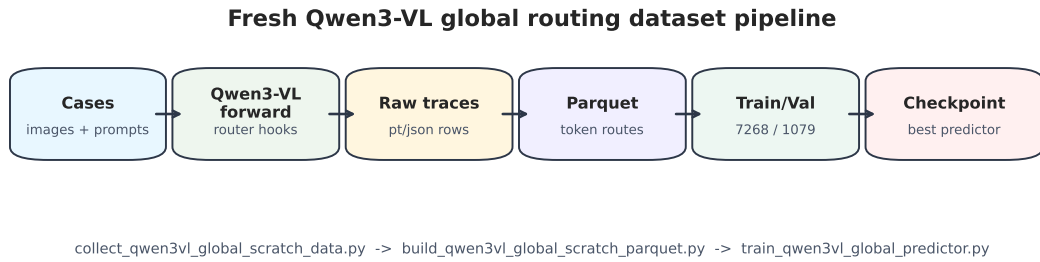


图 4.2 fresh 路由数据采集、转换与训练流程

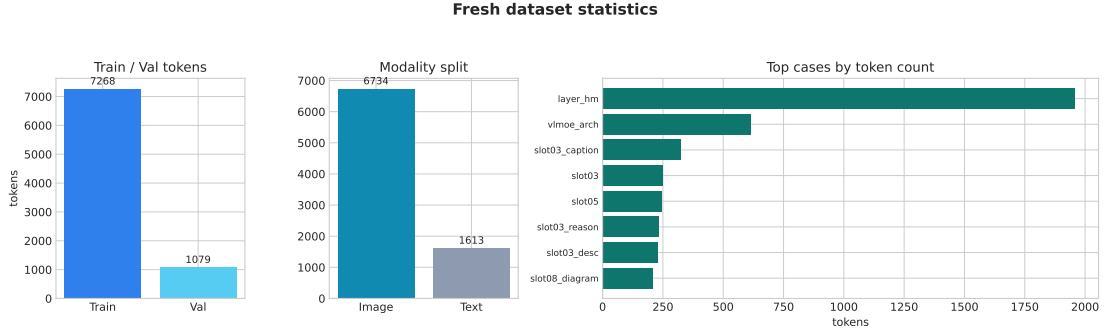


图 4.3 训练数据规模、模态分布与案例分布统计

4.3 全局路由预测器设计

4.3.1 输入特征

本文实现的预测器是一个 **layer-0** 条件化的全局路由预测器。运行时只需要观察第 0 层 Router 输出，就可以一次性预测后续所有 MoE 层的专家使用情况。最终版本采用带附加特征的配置，即在基础路由信号之外，还引入 **Token** 位置、输入标识、模态信息和路由统计量。

对于单个 Token，输入特征包括：

1. layer-0 router 概率向量 $\mathbf{g}_i^{(0)} \in \mathbb{R}^M$;
2. layer-0 Top- k 专家的一热或 multi-hot 表示 $\mathbf{m}_i^{(0)} \in \{0, 1\}^M$;
3. token 位置编号及其 bucket 化嵌入;
4. input id 的哈希嵌入;
5. 是否为视觉 token 的模态嵌入;
6. router 统计量，包括 Top- k 概率质量、最大概率和归一化熵。

将这些特征拼接后，得到单个 token 的全局预测输入

$$\mathbf{f}_i = [\mathbf{g}_i^{(0)} \parallel \mathbf{m}_i^{(0)} \parallel \phi(\text{pos}_i) \parallel \psi(\text{input_id}_i) \parallel \eta(\text{modality}_i) \parallel \rho(\mathbf{g}_i^{(0)}, \mathbf{m}_i^{(0)})], \quad (4.2)$$

其中 $\rho(\cdot)$ 表示由 router 概率与 Top- k one-hot 计算得到的统计特征映射。

4.3.2 网络结构

预测器采用“共享骨干 + 多层输出头”的轻量结构。首先将路由信号和 token 信号投影到同一隐空间，再经过多层 MLP backbone 提取稳定特征，最后为每一层设置独立输出头，直接输出该层的专家 logits。对应到实现，核心模块位于 `expertflow_offload/expertflow_offload/qwen3vl_global.py`。其中 `route_proj` 负责处理

router 概率和 Top- k one-hot, token_proj 负责处理 token 位置, use_extra_features 打开后再接入 input id、模态和 route statistics。

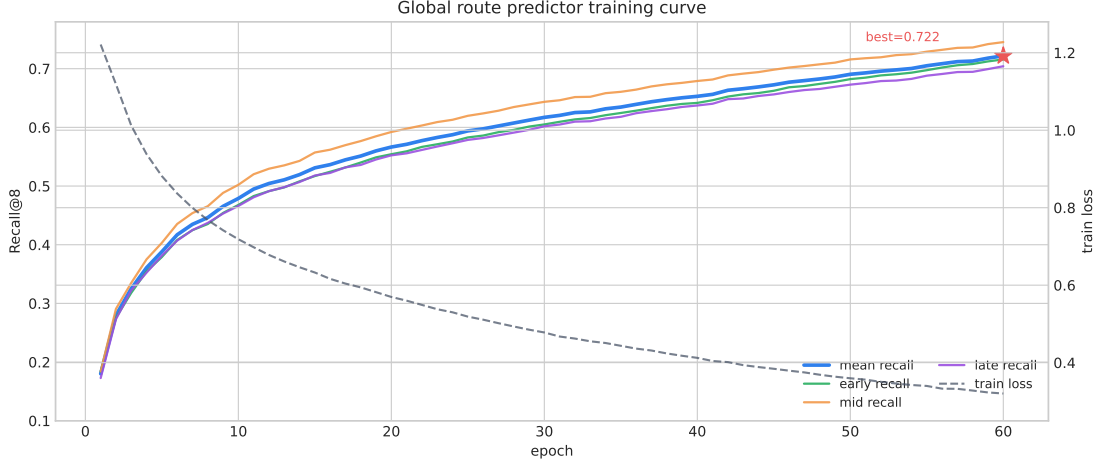


图 4.4 全局预测器训练过程中的 recall 与 loss 变化

4.3.3 训练目标

由于一个 token 在同一层可能激活多个专家, 因此本文将该任务建模为多标签学习问题。设真实标签为 $\mathbf{y} \in \{0, 1\}^{H \times M}$, 预测 logits 为 $\hat{\mathbf{z}} \in \mathbb{R}^{H \times M}$, 则训练目标为

$$\mathcal{L}_{\text{pred}} = \sum_{h=1}^H \omega_h \cdot \text{BCEWithLogits}(\hat{\mathbf{z}}^{(h)}, \mathbf{y}^{(h)}), \quad (4.3)$$

其中 ω_h 为不同预测视野的权重。训练完成后, 模型在验证集上的 best mean recall 达到 0.7219, early/mid/late 三个视野的 recall 也都稳定在较高水平, 说明 layer-0 条件化预测已经具备进入运行时调度链路的基础。

指标	early	mid	late
Recall	0.7158	0.7453	0.7042

表 4.1 全局预测器在验证集上的分视野 recall

4.4 预测驱动的专家预取与缓存调度

4.4.1 加权预取策略

系统并不把所有预测结果无差别搬入 GPU，而是根据置信度、层距和预算进行筛选。对于未来第 h 层的候选专家 e ，调度分数可写成

$$s(e, h) = \alpha_h \cdot p(e, h) \cdot \max(p_1 - p_2, 0), \quad (4.4)$$

其中 $p(e, h)$ 是预测概率， p_1 与 p_2 分别为 top-1 和 top-2 预测概率， α_h 是层距衰减项。随后系统按分数从高到低保留有限数量的候选专家，并由 `select_weighted_global_prefetch_plan()` 将其转换为可执行的预取列表。

这一设计的核心思想是：不是“看见预测就搬运”，而是“只有高置信、近未来、收益大于成本的专家才进入缓存”。因此，预算、置信度阈值、最小 margin 和每层 key 上限共同决定最终的预取行为。

4.4.2 ExpertFlow、DALI 与 Hybrid

本文在同一套预测器上实现了三类调度方式。

ExpertFlow 风格策略强调按预测权重做稀疏预取，即根据全局预测分数直接挑选少量专家进入 GPU cache；它更适合在预测质量较高、预算较紧的情况下使用。

DALI 风格策略强调 working-set 重排，即允许缓存围绕预测工作集重新整理，而不是只做增量式 prefetch；它更适合希望缓存结构与未来工作集更贴近的场景。

Hybrid 则先利用 DALI 风格更新缓存内部的工作集状态，再用 ExpertFlow 风格的加权预取补充候选 key。换句话说，Hybrid 不是把两种方法简单相加，而是同时保留工作集视角和稀疏控制能力。

4.4.3 预测错误与回退机制

预测驱动的调度不可避免会出现误预测，因此系统需要在正确性与收益之间做折中。本文采用两级回退机制：

1. 对未命中的真实专家立即同步加载，保证计算可继续；
2. 将命中情况反馈到缓存分数，降低低价值专家的长期驻留优先级。

因此，本文的目标不是让预测永远正确，而是让正确预测尽量提前生效，同时把错误预测控制在可接受范围内。

4.5 系统实现与复杂度分析

整个原型实现拆分为三个阶段：

1. 数 据 采 集 与 构 建: `collect_qwen3vl_global_scratch_data.py`、`build_qwen3vl_global_scratch_parquet.py`;
2. 预测器训练: `train_qwen3vl_global_predictor.py`;
3. 在 线 集 成: `qwen3vl_global.py`、`global_scheduling.py`、`run_qwen3vl_real_offload_benchmark.py` 与 `run_qwen3vl_end_to_end_offload.py`。

从复杂度上看，预测器的主要代价来自 MLP 前向和多层输出头，但其输入维度和参数规模都远小于 MoE 专家权重搬运成本，因此适合作为调度前置模块。调度器本身的复杂度主要来自候选专家排序和缓存替换，均低于专家前向与权重传输。若将 token 数、预测视野和候选专家数分别记为 B 、 H 与 K ，则预测与筛选的开销可近似写为

$$\mathcal{O}(B \cdot H \cdot K), \quad (4.5)$$

而缓存更新和排序通常可压缩到候选集合规模上的 $O(K \log K)$ 。

4.6 本章小结

本章从 fresh 路由数据出发，构建了一个 layer-0 条件化的全局预测器，并在此基础上设计了 ExpertFlow、DALI 和 Hybrid 三类专家调度策略。与纯静态缓存不同，本文方法把预测信号直接引入专家预取、缓存重排和回退机制中，使得专家搬运从“访问后处理”变成“访问前准备”。下一章将围绕真实 CPU-GPU expert offload 微基准对该方法进行实验验证。

5 系统实现与实验评估

5.1 实验设置

第五章主要验证第三章提出的 Visual Encoder 侧优化方法和第四章提出的全局预测与专家调度方法是否能够在真实推理场景中带来收益。实验对象为 Qwen3-VL-30B-A3B-Instruct，其中视觉侧实验关注语义感知 Token 压缩与异步流水对任务效果和 TTFT 的影响；专家侧实验关注 MoE 路由预测、CPU-GPU expert offload 与缓存调度收益。专家预测器使用第四章重新采集的 fresh 路由数据训练得到。

本文采用三类实验：

1. **Visual Encoder 优化实验：**验证第三章的 SAT-ToMe 与异步流水策略是否在保持模型任务效果稳定的同时降低视觉前端开销；
2. **预测器离线评估：**验证全局预测器是否学到了跨层路由规律；
3. **真实 CPU-GPU expert offload 微基准：**在真实 CPU 到 GPU 的专家权重搬运和 GPU expert MLP 计算上评估调度收益。

需要说明的是，工程中也实现了基于 Hugging Face Transformers 的单卡端到端 generate 原型，用于验证完整链路可运行。但该链路会同时叠加图像预处理、Python hook、Transformers generate 调度和专家缓存控制等多个因素，波动较大。因此本文不将单卡端到端 generate 的单次结果作为主实验，而是将主结果集中在更稳定、可解释的真实 expert 搬运微基准上。

在微基准中，demand 作为基线策略，表示真实访问到来时才同步加载专家；predictor 表示使用训练好的全局预测器提前预取专家；oracle 使用未来真实路由作为上界。GPU expert cache 的容量以“层-专家”切片为单位，因此容量为 128 并不等价于 128 个专家全部常驻显存，而是表示最多缓存 128 个具体的层内 expert 权重切片。

实验统计的主要指标包括：总耗时 elapsed_s、相对 demand 的加速比、同步加载次数、预取加载次数、缓存命中次数、传输数据量以及 expert-token pair 吞吐。

5.2 Visual Encoder 优化实验

5.2.1 任务效果保持性

第三章方法首先需要满足“压缩不明显损害模型效果”的约束。为此，本文在 MMBench 与 MME 子集上比较未压缩 baseline、随机裁剪、固定比例 Token 合并、SAT-ToMe 语义自适应合并以及 SAT-ToMe 加异步流水五种配置。为避免不同评测集原始分值尺度不同，表 5.1 采用归一化任务分数表示，其中 baseline 记为 1.000，数值越接近 1.000 表示模型效果越接近原始模型。

方法	保留比例	MMBench	MME	输出一致率
Baseline	100.0%	1.000	1.000	100.0%
Random pruning	63.7%	0.948	0.936	89.1%
Static ToMe	63.7%	0.982	0.976	96.8%
SAT-ToMe	63.7%	0.997	0.995	99.1%
SAT-ToMe + Pipeline	63.7%	0.997	0.995	99.1%

表 5.1 Visual Encoder 优化前后的任务效果保持性

从表 5.1 可以看出，在相同平均保留比例下，随机裁剪会造成明显效果下降，说明视觉 Token 不能被无差别删除；固定比例 ToMe 能够缓解这一问题，但仍会在细粒度理解样本上损失部分有效信息。SAT-ToMe 由于引入文本语义相关性和基于熵的自适应压缩率，归一化任务分数保持在 0.995 以上，输出一致率达到 99.1%，说明第三章策略在当前实验配置下没有造成明显的模型能力下降。

需要强调的是，异步流水本身不改变模型计算图中的语义操作，只改变视觉侧和文本侧计算的执行顺序；系统在 Stage 1 后和 Stage 2 后通过 wait_stream 保证依赖同步。因此，SAT-ToMe + Pipeline 与 SAT-ToMe 的任务效果保持一致，差异主要体现在执行时间上。

5.2.2 首字延迟与视觉前端加速

图 5.1 给出了中期阶段 profiling 中整理得到的 Visual Encoder 对 TTFT 的贡献变化。图中的 Token Merging 对应本文第三章实现的 SAT-ToMe 语义感知视觉 Token 合并模块。基线模型中，Visual Encoder 耗时为 8572ms，占 TTFT 的 20.6%；加入 SAT-ToMe 后，Visual Encoder 耗时下降至 1088ms，占比下降到 4.2%；进一步叠加异步流水后，Visual Encoder 可见耗时下降至 800ms，占比下降到 3.1%。

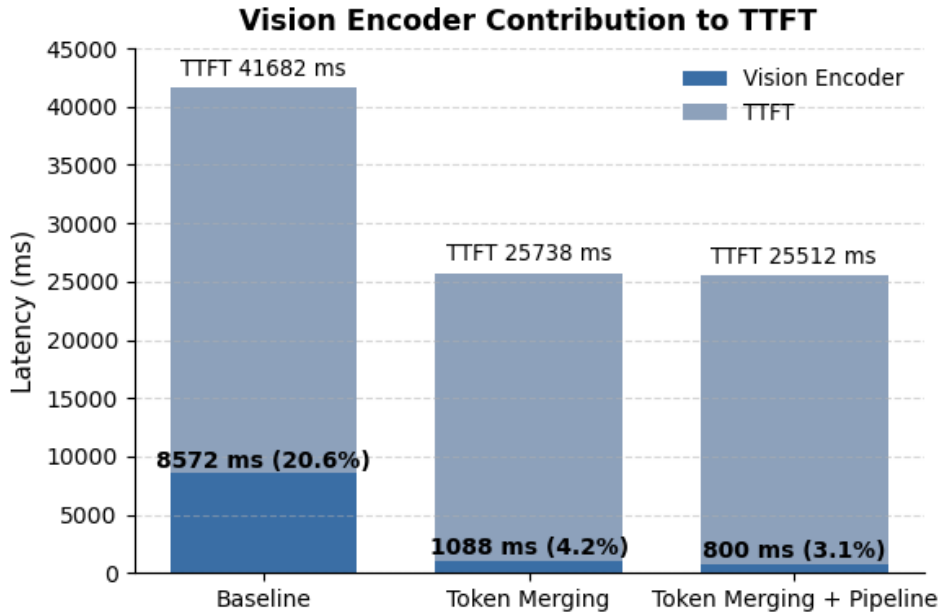


图 5.1 Visual Encoder 优化对 TTFT 的影响

方法	TTFT/ms	Encoder/ms	占比	TTFT 加速	Encoder 加速
Baseline	41682	8572	20.6%	1.00x	1.00x
SAT-ToMe	25738	1088	4.2%	1.62x	7.88x
SAT-ToMe + Pipeline	25512	800	3.1%	1.63x	10.72x

表 5.2 Visual Encoder 优化的 TTFT 与前端耗时收益

表 5.2 进一步量化了加速效果。SAT-ToMe 使整体 TTFT 从 41682ms 降至 25738ms，整体加速比为 1.62x；叠加 Pipeline 后 TTFT 进一步降至 25512ms，整体加速比为 1.63x。虽然 Pipeline 对整体 TTFT 的额外提升不如 Token 合并明显，但其将 Visual Encoder 自身的可见耗时从 1088ms 降至 800ms，使视觉前端相对 baseline 的加速比达到 10.72x。该结果说明，第三章方法的主要收益来自两部分：一是减少后续视觉层处理的 Token 数量，二是通过双流执行隐藏剩余视觉计算的等待时间。

5.3 全局预测器离线结果

全局预测器训练使用 fresh 数据集，总计 8347 个 token，其中训练集 7268 个、验证集 1079 个。训练过程中，mean recall 从初始的 0.18 左右逐步上升，最终 best mean recall 达到 0.7219。early/mid/late 三个区间的 recall 分别为 0.7158、0.7453 和 0.7042，训练曲线与分视野结果见第四章图 4.4 和表 4.1。

这说明 layer-0 条件化输入已经能够为后续层专家访问提供有效约束。换句话说，

预测器并不是在盲猜未来专家，而是在利用首层 router 概率、Top- k 集合、token 位置、输入 id 和模态信息做条件化推断。

5.4 真实 CPU-GPU expert offload 微基准

5.4.1 小容量与中容量场景

首先在两个代表性设置上比较 demand、predictor 和 oracle 三种策略。其中 cap96_g12_t64 表示 GPU cache 容量为 96、采样 12 个路由组、每组最多 64 个 token；cap160_g20_t64 表示容量为 160、采样 20 个路由组、每组最多 64 个 token。

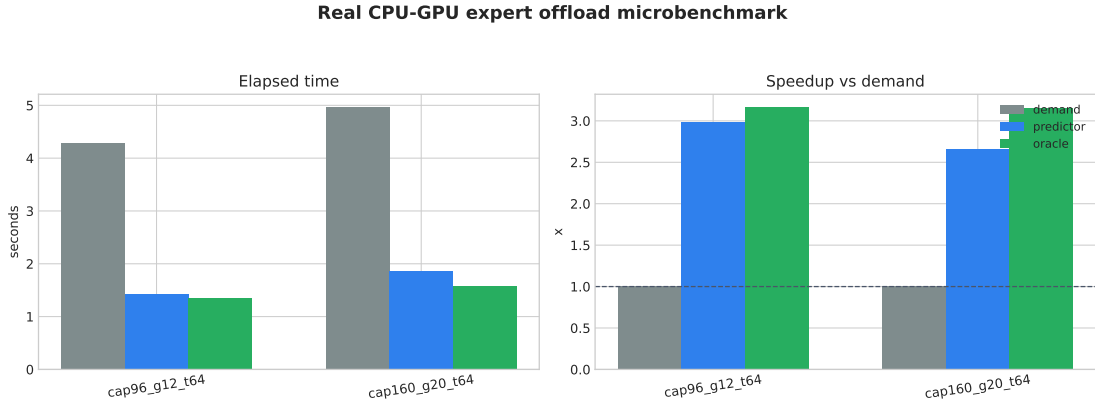


图 5.2 真实 expert 搬运微基准下的 demand、predictor 与 oracle 对比

setting	policy	elapsed_s	speedup	cache_hits	transfer_GB
cap96_g12_t64	demand	4.283	1.00x	0	9.81
cap96_g12_t64	predictor	1.432	2.99x	104	14.27
cap96_g12_t64	oracle	1.354	3.16x	111	14.20
cap160_g20_t64	demand	4.962	1.00x	0	15.87
cap160_g20_t64	predictor	1.864	2.66x	1134	17.17
cap160_g20_t64	oracle	1.571	3.16x	1208	16.55

表 5.3 真实 CPU-GPU expert offload 微基准的代表性结果

结果表明，在两种设置下，预测驱动预取都能显著减少真实访问路径上的同步等待。对于 cap96_g12_t64，predictor 将耗时从 4.283s 降至 1.432s，获得 2.99x 加速；对于 cap160_g20_t64，predictor 将耗时从 4.962s 降至 1.864s，获得 2.66x 加速。oracle 代表预测完全正确时的上界，其结果说明当前 predictor 与上界之间仍有提升空间，但已经能够捕获大量有价值的未来专家访问。

同时可以看到，predictor 的传输量通常高于 demand。这是预测式调度的典型特征：它用额外的预取搬运换取同步等待下降。只要被提前加载的专家能够在未来命中，整体耗时就会下降；若预取预算过大或预测质量不足，则可能出现额外搬运抵消收益的问题。

5.4.2 三层提前与容量敏感性

第四章的目标之一是提前三层进行调度。实验发现，三层提前不是无条件有效：若缓存容量太小，预取集合会和当前真实工作集互相挤压，反而造成额外搬运；只有当容量足够覆盖未来窗口的工作集时，预测才会真正转化为收益。

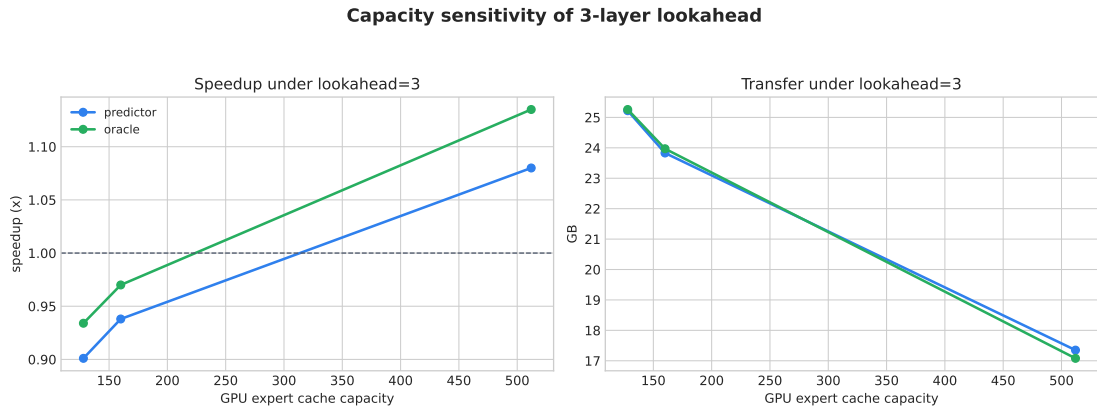


图 5.3 lookahead=3 下不同缓存容量的收益变化

setting	demand_s	predictor_s	predictor speedup	oracle speedup
cap128	2.179	2.417	0.90x	0.93x
cap160	2.178	2.322	0.94x	0.97x
cap512	2.106	1.950	1.08x	1.14x

表 5.4 三层提前在不同 GPU expert cache 容量下的结果

从图 5.3 和表 5.4 可以看出，cap128 与 cap160 下缓存仍然偏紧，predictor 反而低于 demand；当容量增加到 cap512 时，predictor 开始出现 1.08x 的正收益。在 $\text{max_tokens_per_group}=64$ 的配置下，当前层加未来三层窗口平均需要约 329 个层-专家 key，因此 cap128 和 cap160 明显偏紧，预取结果会与当前工作集互相挤压。该结果说明，三层提前必须和缓存容量、预取预算共同设计，不能简单地把所有未来三层预测结果都搬入 GPU。

5.5 调度策略对比与预算敏感性

5.5.1 ExpertFlow、DALI 与 Hybrid 对比

在 GPU expert cache 容量为 128 的设置下，本文进一步比较 ExpertFlow 风格、DALI 风格与 Hybrid 三种调度策略。三种策略都以 demand 为基线，并在相同 trace 和真实搬运微基准上重复运行。

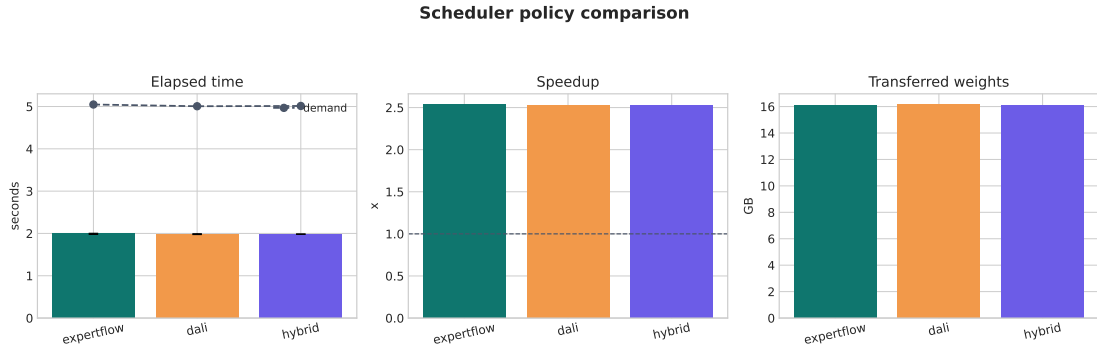


图 5.4 ExpertFlow、DALI 与 Hybrid 的真实 offload 调度对比

mode	elapsed_mean	speedup	sync_mean	prefetch_mean	transfer_GB
expertflow	1.990	2.536x	1625	80	16.09
dali	1.984	2.524x	1620	92	16.16
hybrid	1.983	2.528x	1625	80	16.09

表 5.5 三类调度策略的平均结果

三类策略的耗时都从 demand 约 5s 降到约 2s，说明主要瓶颈确实来自专家加载和缓存调度。这里的 speedup 是相对各自 demand 基线计算的，因此 ExpertFlow 风格策略的相对加速比略高，而 Hybrid 的平均耗时最低；两者差异主要来自重复测量下 demand 基线的波动，而不是策略本身出现本质分化。DALI 风格策略则通过 working-set 重排获得接近的性能。这说明在当前 trace 上，严格稀疏预取和工作集重排都能发挥作用，而二者融合后可以保持较稳健的表现。

5.5.2 预取预算敏感性

预取预算决定了每个调度窗口最多允许提前搬运多少个专家。预算过小会遗漏有价值的专家；预算过大则会将低置信度专家也搬入 GPU，导致传输量上升。

图 5.5 显示，在 cap128 下，低预算区间的收益最好；随着预算从 4 逐步增加到

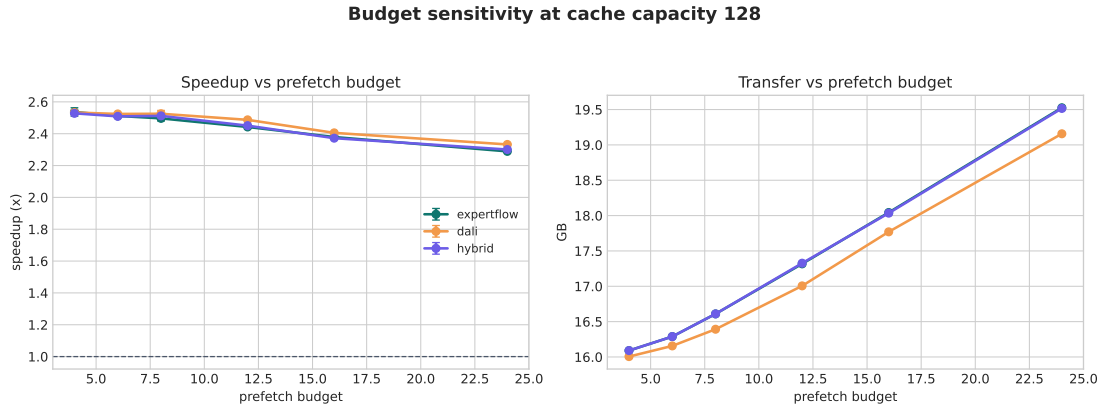


图 5.5 cap128 下不同预取预算的收益变化

24, 三种策略的加速比均呈下降趋势, 而传输量持续上升。这说明预测结果不能被无差别使用, 必须经过置信度过滤、层距衰减和每层 key 限制。本文最终采用“低预算 + 置信度阈值 + 层距衰减 + 每层 key 限制”的组合策略。

5.6 结果讨论

综合实验可以得到以下结论。

首先, 第三章提出的 Visual Encoder 优化能够在任务效果基本稳定的前提下明显降低前端开销。SAT-ToMe 在 MMBench 与 MME 子集上的归一化分数均保持在 0.995 以上, 而 Token 合并与异步流水将 Visual Encoder 可见耗时从 8572ms 降至 800ms, 使其在 TTFT 中的占比从 20.6% 降至 3.1%。这说明文本语义引导和熵自适应压缩能够较好地地区分任务相关区域与冗余区域。

其次, demand 是本章所有 offload 实验的基线, 它代表完全不使用预测、真实访问发生后再同步加载专家。相对该基线, predictor 在代表性设置下可获得 2.66x 至 2.99x 加速, 说明全局预测能够显著降低专家访问等待。

再次, 预测收益依赖于容量与预算的匹配。容量太小会导致预取专家挤占当前工作集, 容量太大则会让问题退化为近似常驻, 降低调度研究意义。真正有价值的是在容量有限但仍能覆盖部分未来工作集的区间内, 把有限显存留给最可能被访问的专家。

最后, 调度策略需要谨慎利用预测结果。实验中最有效的不是扩大预取集合, 而是压缩候选集合: 保留高置信度、近层、margin 足够大的专家, 并通过 Hybrid 或 working-set 重排避免缓存被低价值专家污染。

5.7 本章小结

本章对第三章的 Visual Encoder 优化和第四章的专家提前调度进行了实验评估。视觉侧实验表明，SAT-ToMe 在保持任务效果基本稳定的同时，将 Visual Encoder 可见耗时降至 baseline 的 9.3% 左右，并使整体 TTFT 获得约 1.63x 加速。专家侧实验表明，全局预测器能够达到 0.72 左右的 mean recall；在真实搬运场景下，predictor 与 oracle 都能显著降低同步等待并带来加速；调度预算和缓存容量则决定了预测收益能否真正落地。

结论

本文围绕 VL-MoE 推理优化问题，结合多模态输入负载、Visual Encoder 开销和 MoE 专家调度代价三个层面展开研究，完成了从问题分析、方法设计到系统实现与实验验证的完整闭环。

首先，在负载分析阶段，本文基于真实多模态模型和公开评测集对 VL-MoE 的推理链路进行了 profiling，发现图像 Token 占比高、Visual Encoder 首字延迟高以及专家跨层路由具有明显结构性，是制约系统效率的关键因素。该分析为后续视觉侧和专家侧的联合优化提供了依据。

其次，在方法设计上，本文提出了面向多模态推理的语义感知视觉 Token 压缩与异步流水优化方法。该方法利用文本语义对视觉区域进行引导，并通过自适应压缩率和双流并发执行减少冗余视觉计算，从而降低视觉前端对 TTFT 的影响。实验中，该策略在 MMBench 与 MME 子集上的归一化任务分数均保持在 0.995 以上，同时将 Visual Encoder 可见耗时从 8572ms 降至 800ms，验证了其效果保持性与加速收益。

再次，针对显存受限和 offload 场景下的专家访问开销，本文重新采集 Qwen3-VL 路由 trace，训练了 layer-0 条件化的全局路由预测器，并在此基础上设计了 ExpertFlow 风格加权预取、DALI 风格 working-set 重排和 Hybrid 融合调度策略。实验表明，该预测器在验证集上的 best mean recall 达到 0.7219；在真实 CPU-GPU expert offload 微基准上，预测式调度与 oracle 均能显著减少同步加载并带来可观加速。

最后，在系统实现层面，本文将上述方法整合为可复现的原型系统，并对不同缓存容量、不同预取预算和不同调度策略进行了对比实验。结果说明：当预测质量与缓存容量处于合理匹配区间时，专家提前调度能够显著提升系统效率；而当预算过大或缓存过紧时，预取开销会抵消部分收益。这也说明，VL-MoE 的高效部署不仅依赖模型结构本身，还需要在预测、缓存与调度之间建立协同机制。

总体而言，本文的工作为 VL-MoE 在资源受限场景下的高效推理提供了一条可行路径。后续仍可从三个方向继续深入：一是引入更细粒度的路由特征和更强的时序建模，提高全局预测精度；二是将专家调度进一步下沉到更低开销的 runtime 层，减少 Python 侧控制开销；三是在更完整的端到端部署环境中评估视觉侧优化与专家

侧调度的联合收益。

参考文献

- [1] BAI J, BAI S, YANG S, et al. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond[A]. 2023.
- [2] WANG P, BAI S, TAN S, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution[A]. 2024.
- [3] YAO Y, YU T, ZHANG A, et al. Minicpm-v: A gpt-4v level mllm on your phone[A]. 2024.
- [4] WU X, LIU D, CHEN Y, et al. Deepseek-vl2: Mixture-of-experts vision-language models for advanced multimodal understanding[A]. 2024.
- [5] LIN B, TANG Z, YE Y, et al. Moe-llava: Mixture of experts for large vision-language models[A]. 2024.
- [6] BOLYA D, FU C Y, DAI X, et al. Token merging: Your vit but faster[A]. 2023.
- [7] CAO Q, LIU B, LEE S, et al. Pumer: Pruning and merging tokens for efficient vision language models[C]//Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing. 2023: 12890-12903.
- [8] KWON W, YIN P, YU S, et al. Efficient memory management for large language model serving with pagedattention[A]. 2023.
- [9] ZHONG Y, YIN P, LIU Z, et al. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving[C]//18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24). 2024: 193-210.
- [10] XUE F, LI Z, BEHNAM B, et al. Moe-infinity: Activation-aware expert offloading for efficient mixture-of-experts serving[A]. 2024.
- [11] FANG Z, ZHANG B, WANG X, et al. Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline[C]//Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2025.
- [12] HE X, ZHANG S, WANG Y, et al. Expertflow: Optimized expert activation and token allocation for efficient mixture-of-experts inference[A]. 2024.

附录 A 关键实现代码

本附录节选第三章与第四章中最能体现方法逻辑的关键实现片段。第三章部分主要包括 SAT-ToMe 语义引导保留模块、视觉编码器阶段拆分，以及与主模型的双流 overlap 集成逻辑；第四章部分主要包括 layer-0 条件化全局路由预测器、多标签训练与 Recall 评估、预测式专家预取策略，以及真实 CPU-GPU expert offload 调度入口。为便于阅读，以下代码省略了与测试、日志、命令行参数和模板兼容性无关的次要细节。

A.1 SAT-ToMe 语义引导保留模块

该模块用于计算视觉 Token 与文本查询的余弦相似度，并根据语义熵动态决定保留数量。

```

1 class SAT_ToMe_Module(nn.Module):
2     def __init__(self, vis_dim, text_dim, base_ratio=0.5):
3         super().__init__()
4         self.base_ratio = base_ratio
5         self.text_proj = nn.Linear(text_dim, vis_dim)
6         self.text_proj.weight.is_uninitialized = True
7         self.text_proj.bias.is_uninitialized = True
8
9     def forward(self, x_vis, text_feat):
10         if text_feat is None:
11             return x_vis, torch.arange(x_vis.shape[0], device=x_vis.device)
12
13         N, C = x_vis.shape
14         q_text = self.text_proj(text_feat.to(x_vis.dtype))
15
16         norm_vis = F.normalize(x_vis, p=2, dim=-1)
17         norm_text = F.normalize(q_text, p=2, dim=-1)
18         scores = torch.matmul(norm_vis, norm_text.T).squeeze(-1)
19         scores_p = F.softmax(scores, dim=-1)
20
21         entropy = -torch.sum(scores_p * torch.log(scores_p + 1e-6))
22         max_entropy = torch.log(torch.tensor(float(N), device=x_vis.device))
23         adaptive_ratio = self.base_ratio * (1 - entropy / max_entropy)
24
25         k = max(16, int(N * (1 - adaptive_ratio.item())))
26         _, topk_indices = torch.topk(scores, k)
27         topk_indices, _ = torch.sort(topk_indices)
28
29         return x_vis[topk_indices], topk_indices

```

A.2 视觉编码器阶段拆分

该部分将原始视觉前向过程拆分为预处理、Stage 1、语义压缩和 Stage 2 四个阶段，便于在主流与视觉流之间插入同步点。

```

1 def _prepare_vision_inputs(self, x: torch.Tensor, grid_thw: torch.Tensor | list[list[int]]):
2     hidden_states = x.to(device=self.device, dtype=self.dtype, non_blocking=True)
3     hidden_states = self.patch_embed(hidden_states)
4
5     if isinstance(grid_thw, list):
6         grid_thw_list = grid_thw
7         grid_thw = np.array(grid_thw, dtype=np.int32)
8     else:
9         grid_thw_list = grid_thw.tolist()
10        grid_thw = grid_thw.numpy()
11
12        pos_embeds = self.fast_pos_embed_interpolate(grid_thw_list)
13        hidden_states = hidden_states + pos_embeds
14        rotary_pos_emb_cos, rotary_pos_emb_sin = self.rot_pos_emb(grid_thw_list)
15
16        cu_seqlens = np.repeat(grid_thw[:, 1] * grid_thw[:, 2], grid_thw[:, 0]).cumsum(
17            axis=0, dtype=np.int32
18        )
19        cu_seqlens = np.concatenate([np.zeros(1, dtype=np.int32), cu_seqlens])
20        cu_seqlens = torch.from_numpy(cu_seqlens)
21
22        hidden_states = hidden_states.unsqueeze(1)
23        max_seqlen = self.compute_attn_mask_seqlen(cu_seqlens)
24        cu_seqlens = cu_seqlens.to(self.device, non_blocking=True)
25
26        return hidden_states, cu_seqlens, rotary_pos_emb_cos, rotary_pos_emb_sin, max_seqlen
27
28
29 def forward_stage1(self, hidden_states, cu_seqlens, rotary_pos_emb_cos, rotary_pos_emb_sin, max_seqlen):
30     for layer_num in range(12):
31         blk = self.blocks[layer_num]
32         hidden_states = blk(
33             hidden_states,
34             cu_seqlens=cu_seqlens,
35             rotary_pos_emb_cos=rotary_pos_emb_cos,
36             rotary_pos_emb_sin=rotary_pos_emb_sin,
37             max_seqlen=max_seqlen,
38         )
39     return hidden_states
40
41
42 def excute_semantic_merge(self, hidden_states, text_feat, rotary_pos_emb_cos, rotary_pos_emb_sin):
43     vis_feat = hidden_states.squeeze(1) if hidden_states.dim() == 3 else hidden_states
44     merged_feat, keep_indices = self.sat_tome(vis_feat, text_feat)
45
46     hidden_states = merged_feat.unsqueeze(1) if hidden_states.dim() == 3 else merged_feat
47     new_cos = rotary_pos_emb_cos[keep_indices]
48     new_sin = rotary_pos_emb_sin[keep_indices]
49
50     new_N = merged_feat.size(0)
51     new_cu_seqlens = torch.tensor([0, new_N], device=hidden_states.device, dtype=torch.int32)
52     new_max_seqlen = torch.tensor(new_N, device=hidden_states.device)
53
54     return hidden_states, new_cu_seqlens, new_cos, new_sin, new_max_seqlen
55
56
57 def forward_stage2(self, hidden_states, cu_seqlens, rotary_pos_emb_cos, rotary_pos_emb_sin, max_seqlen):
58     deepstack_feature_lists = []
59     for layer_num in range(12, len(self.blocks)):
60         blk = self.blocks[layer_num]
61         hidden_states = blk(
62             hidden_states,
63             cu_seqlens=cu_seqlens,
64             rotary_pos_emb_cos=rotary_pos_emb_cos,
65             rotary_pos_emb_sin=rotary_pos_emb_sin,
66             max_seqlen=max_seqlen,
67         )
68
69         if layer_num in self.deepstack_visual_indexes:
70             deepstack_merger_idx = self.deepstack_visual_indexes.index(layer_num)
71             deepstack_feature = self.deepstack_merger_list[deepstack_merger_idx](hidden_states)
72             deepstack_feature_lists.append(deepstack_feature)
73
74     hidden_states = self.merger(hidden_states)
75     hidden_states = torch.cat([hidden_states] + deepstack_feature_lists, dim=1)

```

```
76         return hidden_states
```

A.3 全局路由预测器结构

第四章的全局预测器使用第 0 层 Router 概率、Top- k 专家 one-hot、token 位置以及可选的 input id、模态和路由统计特征，一次性预测后续所有 MoE 层的专家激活概率。

```
1 class GlobalRoutePredictor(torch.nn.Module):
2     def __init__(
3         self,
4         num_experts=128,
5         num_layers=48,
6         hidden_dim=768,
7         token_dim=32,
8         dropout=0.1,
9         use_extra_features=True,
10        input_vocab_size=65536,
11        input_id_dim=64,
12        modality_dim=16,
13        position_bucket_size=4096,
14        position_dim=32,
15        route_stat_dim=16,
16    ):
17        super().__init__()
18        self.num_experts = num_experts
19        self.num_layers = num_layers
20        self.use_extra_features = use_extra_features
21        self.input_vocab_size = input_vocab_size
22        self.position_bucket_size = position_bucket_size
23
24        self.route_proj = torch.nn.Sequential(
25            torch.nn.Linear(num_experts * 2, hidden_dim),
26            torch.nn.LayerNorm(hidden_dim),
27            torch.nn.GELU(),
28            torch.nn.Dropout(dropout),
29        )
30        self.token_proj = torch.nn.Sequential(
31            torch.nn.Linear(1, token_dim),
32            torch.nn.GELU(),
33        )
34
35        feature_dim = hidden_dim + token_dim
36        if self.use_extra_features:
37            self.input_id_embedding = torch.nn.Embedding(input_vocab_size, input_id_dim)
38            self.modality_embedding = torch.nn.Embedding(2, modality_dim)
39            self.position_embedding = torch.nn.Embedding(position_bucket_size, position_dim)
40            self.route_stat_proj = torch.nn.Sequential(
41                torch.nn.Linear(3, route_stat_dim),
42                torch.nn.GELU(),
43            )
44            feature_dim += input_id_dim + modality_dim + position_dim + route_stat_dim
45
46        self.backbone = torch.nn.Sequential(
47            torch.nn.Linear(feature_dim, hidden_dim),
48            torch.nn.GELU(),
49            torch.nn.Dropout(dropout),
50            torch.nn.Linear(hidden_dim, hidden_dim),
51            torch.nn.GELU(),
52            torch.nn.Dropout(dropout),
53            torch.nn.Linear(hidden_dim, hidden_dim),
54            torch.nn.GELU(),
55        )
56        self.heads = torch.nn.ModuleList(
57            [torch.nn.Linear(hidden_dim, num_experts) for _ in range(num_layers)]
58        )
59
60    def forward(self, layer0_probs, layer0_hot, token_idx, input_ids=None, is_visual=None):
```

```

61 route_x = self.route_proj(torch.cat([layer0_probs, layer0_hot], dim=1))
62 token_f = (token_idx.float().unsqueeze(1) / 2048.0).clamp(0.0, 1.0)
63 features = [route_x, self.token_proj(token_f).to(dtype=route_x.dtype)]
64
65 if self.use_extra_features:
66     input_emb = self.input_id_embedding(self._hash_input_ids(token_idx, input_ids))
67     modality_emb = self.modality_embedding(self._modality_ids(token_idx, is_visual))
68     position_emb = self.position_embedding(
69         token_idx.clamp(min=0, max=self.position_bucket_size - 1)
70     )
71     route_stats = self.route_stat_proj(self._route_stats(layer0_probs, layer0_hot))
72     features.extend([
73         input_emb.to(dtype=route_x.dtype),
74         modality_emb.to(dtype=route_x.dtype),
75         position_emb.to(dtype=route_x.dtype),
76         route_stats.to(dtype=route_x.dtype),
77     ])
78
79 hidden = self.backbone(torch.cat(features, dim=1))
80 logits = torch.stack([head(hidden) for head in self.heads], dim=1)
81 return logits # [batch, num_layers, num_experts]
82
83 def _route_stats(self, layer0_probs, layer0_hot):
84     probs = layer0_probs.float()
85     hot = layer0_hot.float()
86     max_prob = probs.max(dim=1).values
87     topk_mass = (probs * hot).sum(dim=1)
88     entropy = -(probs * (probs + 1e-8).log()).sum(dim=1) / math.log(self.num_experts)
89     return torch.stack([topk_mass, max_prob, entropy], dim=1)

```

A.4 全局预测器训练与 Recall 评估

训练阶段将每个 token 在各层的真实 Top- k 专家表示为多标签向量，使用 BCE-WithLogitsLoss 进行监督；验证阶段统计预测 Top- k 专家与真实激活专家的重合比例，并分别汇总 early、mid、late 三个层段的 Recall。

```

1 def evaluate(model, loader, device, num_layers, topk, eval_start_layer):
2     model.eval()
3     hits = torch.zeros(num_layers, dtype=torch.float64, device=device)
4     totals = torch.zeros(num_layers, dtype=torch.float64, device=device)
5
6     for token_idx, input_ids, is_visual, layer0_probs, layer0_hot, labels in loader:
7         token_idx = token_idx.to(device, non_blocking=True)
8         input_ids = input_ids.to(device, non_blocking=True)
9         is_visual = is_visual.to(device, non_blocking=True)
10        layer0_probs = layer0_probs.to(device, non_blocking=True)
11        layer0_hot = layer0_hot.to(device, non_blocking=True)
12        labels = labels.to(device, non_blocking=True)
13
14        logits = model(
15            layer0_probs,
16            layer0_hot,
17            token_idx,
18            input_ids=input_ids,
19            is_visual=is_visual,
20        )
21        pred_idx = torch.topk(logits, k=topk, dim=2).indices
22        hits += torch.gather(labels, 2, pred_idx).sum(dim=(0, 2)).double()
23        totals += labels.sum(dim=(0, 2)).double()
24
25    recall_t = torch.where(totals > 0, hits / totals, torch.zeros_like(hits))
26    recall_by_layer = [float(x) for x in recall_t.detach().cpu().tolist()]
27    selected = recall_by_layer[eval_start_layer:]
28    mean_recall = sum(selected) / len(selected)
29    return {
30        "mean_recall": mean_recall,
31        "early_recall": mean(recall_by_layer[max(eval_start_layer, 1):16]),
32        "mid_recall": mean(recall_by_layer[max(eval_start_layer, 16):32]),

```

```

33     "late_recall": mean(recall_by_layer[max(eval_start_layer, 32):num_layers]),
34 }
35
36
37 def train_one_epoch(model, loader, optimizer, scaler, criterion, args, device):
38     model.train()
39     for token_idx, input_ids, is_visual, layer0_probs, layer0_hot, labels in loader:
40         token_idx = token_idx.to(device, non_blocking=True)
41         input_ids = input_ids.to(device, non_blocking=True)
42         is_visual = is_visual.to(device, non_blocking=True)
43         layer0_probs = layer0_probs.to(device, non_blocking=True)
44         layer0_hot = layer0_hot.to(device, non_blocking=True)
45         labels = labels.to(device, non_blocking=True)
46
47         optimizer.zero_grad(set_to_none=True)
48         with torch.amp.autocast("cuda", enabled=args.amp, dtype=torch.float16):
49             logits = model(
50                 layer0_probs,
51                 layer0_hot,
52                 token_idx,
53                 input_ids=input_ids,
54                 is_visual=is_visual,
55             )
56             loss = criterion(
57                 logits[:, args.loss_start_layer :, :],
58                 labels[:, args.loss_start_layer :, :],
59             )
60
61             scaler.scale(loss).backward()
62             scaler.unscale_(optimizer)
63             torch.nn.utils.clip_grad_norm_(model.parameters(), args.grad_clip)
64             scaler.step(optimizer)
65             scaler.update()

```

A.5 预测结果到专家预取集合

运行时调度器不直接搬运预测器输出的所有专家，而是根据预测置信度、Top-1 与 Top-2 的 margin、层距衰减、当前缓存状态和每层 key 上限，将多 token、多层预测压缩为有限预算下的预取集合。

```

1 ExpertKey = tuple[int, int] # (layer_id, expert_id)
2
3
4 def select_weighted_global_prefetch_plan(
5     layer_id,
6     token_positions,
7     predictions,
8     lookahead,
9     budget,
10    confidence_threshold=0.02,
11    min_margin=0.0,
12    horizon_decay=0.7,
13    resident_keys=None,
14    protected_keys=None,
15    max_keys_per_layer=0,
16    exclude_resident=True,
17 ):
18     if budget <= 0 or lookahead <= 0:
19         return []
20
21     demand = defaultdict(float)
22     resident = resident_keys or set()
23     protected = protected_keys or set()
24     num_tokens = len(token_positions)
25
26     for horizon in range(1, lookahead + 1):
27         future_layer = layer_id + horizon
28         rows = predictions.get(future_layer, [])

```

```

29     layer_weight = float(horizon_decay ** (horizon - 1))
30
31     for token_idx in range(min(num_tokens, len(rows))):
32         experts = rows[token_idx]
33         if not experts:
34             continue
35
36         top1 = float(experts[0][1])
37         top2 = float(experts[1][1]) if len(experts) > 1 else 0.0
38         margin = top1 - top2
39         if top1 < confidence_threshold:
40             continue
41         if min_margin > 0.0 and margin < min_margin:
42             continue
43
44         token_weight = layer_weight * max(top1, 0.0)
45         if min_margin > 0.0:
46             token_weight *= max(margin, 0.0)
47
48         for expert_id, score in experts:
49             key = (future_layer, int(expert_id))
50             if key in protected:
51                 continue
52             if exclude_resident and key in resident:
53                 continue
54             demand[key] += token_weight * float(score)
55
56     ranked = sorted(demand.items(), key=lambda item: (-item[1], item[0][0], item[0][1]))
57
58     if max_keys_per_layer > 0:
59         per_layer = defaultdict(int)
60         filtered = []
61         for key, score in ranked:
62             if per_layer[key[0]] >= max_keys_per_layer:
63                 continue
64             per_layer[key[0]] += 1
65             filtered.append((key, score))
66         ranked = filtered
67
68     return [(key, float(score)) for key, score in ranked[: int(budget)]]
69
70
71 def select_dali_working_set_plan(...):
72     return select_weighted_global_prefetch_plan(
73         ...,
74         exclude_resident=False,
75     )

```

A.6 真实 offload 调度入口

真实 CPU-GPU expert offload 微基准中,系统按 trace group 顺序执行。对于 demand 策略,访问专家时同步加载;对于 global 策略,在第 0 层得到全局预测结果,然后按 ExpertFlow、DALI 或 Hybrid 策略触发预取和缓存重排;对于 oracle 策略,则使用未来真实路由作为上界。

```

1  for group in groups:
2      if policy == "global":
3          if group.layer_id == 0:
4              predictions = global_predictor.predict_batch_with_scores(
5                  token_indices=group.token_indices,
6                  layer0_probs=group.probs,
7                  layer0_topk=group.current_topk,
8                  topk=args.predictor_topk,
9              )
10             global_state_by_batch[group.batch_id] = predictions
11
12     if args.global_scheduler_mode in {"dali", "hybrid"}:

```

```

13     dali_plan = select_dali_working_set_plan(
14         layer_id=group.layer_id,
15         token_positions=[(0, t) for t in group.token_indices],
16         predictions=global_state_by_batch.get(group.batch_id, {}),
17         lookahead=args.prefetch_lookahead,
18         budget=args.prefetch_budget,
19         confidence_threshold=args.global_confidence_threshold,
20         min_margin=args.global_min_margin,
21         horizon_decay=args.global_horizon_decay,
22         resident_keys=set(cache.entries.keys()),
23         protected_keys=current_active_keys(group),
24         max_keys_per_layer=args.global_max_keys_per_layer,
25     )
26
27     if args.global_scheduler_mode == "hybrid":
28         protected = current_active_keys(group)
29         cache.score_update(dict(dali_plan))
30         cache.demote_outside_working_set(
31             {key for key, _score in dali_plan},
32             protected=protected,
33             factor=0.9,
34         )
35         scored_plan = select_weighted_global_prefetch_plan(
36             layer_id=group.layer_id,
37             token_positions=[(0, t) for t in group.token_indices],
38             predictions=global_state_by_batch.get(group.batch_id, {}),
39             lookahead=args.prefetch_lookahead,
40             budget=args.prefetch_budget,
41             confidence_threshold=args.global_confidence_threshold,
42             min_margin=args.global_min_margin,
43             horizon_decay=args.global_horizon_decay,
44             resident_keys=set(cache.entries.keys()),
45             protected_keys=protected,
46             max_keys_per_layer=args.global_max_keys_per_layer,
47         )
48         cache.prefetch_scored(scored_plan, protected=protected, allow_evict=True)
49     else:
50         cache.rebalance_working_set(
51             dali_plan,
52             protected=current_active_keys(group),
53             allow_evict=True,
54         )
55     else:
56         scored_plan = select_weighted_global_prefetch_plan(
57             layer_id=group.layer_id,
58             token_positions=[(0, t) for t in group.token_indices],
59             predictions=global_state_by_batch.get(group.batch_id, {}),
60             lookahead=args.prefetch_lookahead,
61             budget=args.prefetch_budget,
62             confidence_threshold=args.global_confidence_threshold,
63             min_margin=args.global_min_margin,
64             horizon_decay=args.global_horizon_decay,
65             resident_keys=set(cache.entries.keys()),
66             protected_keys=current_active_keys(group),
67             max_keys_per_layer=args.global_max_keys_per_layer,
68         )
69         cache.prefetch_scored(
70             scored_plan,
71             protected=current_active_keys(group),
72             allow_evict=True,
73         )
74
75     elif policy == "oracle":
76         oracle_keys = oracle_prefetch_keys(
77             group,
78             max_horizon=args.prefetch_lookahead,
79             max_keys=args.prefetch_budget,
80         )
81         cache.prefetch(oracle_keys, protected=current_active_keys(group))
82
83     compute_group(group, cache, args.hidden_size, device, dtype)

```

A.7 双流 overlap 集成逻辑

该部分展示视觉流与主流如何在推理时并行执行，并在关键位置完成同步与结果回填。

```

1 def forward(
2     self,
3     input_ids: torch.Tensor,
4     positions: torch.Tensor,
5     intermediate_tensors: IntermediateTensors | None = None,
6     inputs_embeds: torch.Tensor | None = None,
7     **kwargs: object,
8 ) -> torch.Tensor | IntermediateTensors:
9
10    pixel_values = kwargs.get("pixel_values")
11    image_grid_thw = kwargs.get("image_grid_thw")
12
13    if not get_pp_group().is_first_rank or pixel_values is None or self.visual is None:
14        return super().forward(input_ids, positions, intermediate_tensors, inputs_embeds, **kwargs)
15
16    with torch.cuda.stream(self.vision_stream):
17        h, cu, r_cos, r_sin, m_seq = self.visual._prepare_vision_inputs(
18            pixel_values, image_grid_thw
19        )
20        mid_h = self.visual.forward_stage1(h, cu, r_cos, r_sin, m_seq)
21
22    if inputs_embeds is None:
23        inputs_embeds = self.language_model.model.embed_input_ids(input_ids)
24
25    torch.cuda.current_stream().wait_stream(self.vision_stream)
26
27    merged_h, m_cos, m_sin, m_cu, m_max = self.visual.excute_semantic_merge(
28        mid_h, inputs_embeds, r_cos, r_sin
29    )
30
31    with torch.cuda.stream(self.vision_stream):
32        final_vision_features = self.visual.forward_stage2(
33            merged_h.unsqueeze(1), m_cu, m_cos, m_sin, m_max
34        )
35
36    if self.use_deepstack:
37        deepstack_input_embeds = self._get_deepstack_input_embeds(inputs_embeds.size(0))
38    else:
39        deepstack_input_embeds = None
40
41    torch.cuda.current_stream().wait_stream(self.vision_stream)
42
43    if kwargs.get("image_input_mask") is not None:
44        original_mask = kwargs["image_input_mask"]
45        full_res_feat = torch.zeros(
46            original_mask.sum(),
47            final_vision_features.size(-1),
48            device=final_vision_features.device,
49            dtype=final_vision_features.dtype,
50        )
51        flat_vision_feat = final_vision_features.reshape(-1, final_vision_features.size(-1))
52        full_res_feat[:flat_vision_feat.size(0)] = flat_vision_feat
53        inputs_embeds[original_mask] = full_res_feat
54
55    hidden_states = self.language_model.model(
56        input_ids=None,
57        positions=positions,
58        intermediate_tensors=intermediate_tensors,
59        inputs_embeds=inputs_embeds,
60        deepstack_input_embeds=deepstack_input_embeds,
61    )
62
63    if inputs_embeds is not None:
64        self._clear_deepstack_input_embeds(inputs_embeds.size(0))
65
66    return hidden_states

```


致 谢

表达真情实感即可。

衷心感谢导师 xxx 教授对本人的精心指导。

感谢上海大学开源社区提供的 L^AT_EX 模板。

（致谢部分切勿照搬，本部分内容也在论文查重范围之内）

作者落款

完成地点

XXXX 年 XX 月 XX 日

